

IMAGES, FAST

PIXELS AND ADJACENCY · 2-D CONVOLUTION · HISTOGRAMS · SPATIAL FILTERING

Twelve chapters, each a screen or two, in the order the marks fall. Nothing in this paper is descriptive — every question wants a number, a mask or a drawing — so every chapter here puts real numbers through the steps. Each one ends with five lines that are the whole chapter, and the switch in the margin hides everything except those lines. **That is the last-hour version.**

12	48	19	64
CHAPTERS	SECTIONS	DIAGRAMS	SUMMARY LINES

CHAPTER 01

THE DIGITAL IMAGE: SAMPLING, QUANTISATION, RESOLUTION

A picture becomes a file in two steps, and they are independent of each other. Sampling decides how many boxes you cut the picture into. Quantisation decides how many shades each box is allowed. This chapter settles the two resolutions, the storage formula, and the two ugly artefacts that appear when you cut each one too far.

1.1 What a digital image is

COMPUTED · TOOLS/MAKE-FIGURES.PY

Top row: too few samples and the detail goes blocky. Bottom row: too few levels and smooth shading breaks into bands — two different axes, two different failures.



The same photograph down two different axes. Along the top, fewer and fewer samples — the detail goes blocky. Down the bottom, fewer and fewer grey levels — smooth shading breaks into bands. Two different failures, and the exam asks you to tell them apart.

A monochrome image is a two-dimensional function $f(x, y)$. The value at any point is the **intensity** or **gray level** there, and it is always finite and positive. A *digital* image is that function after both its coordinates and its amplitude have been made discrete.

$f(x, y)$ x, y = position (spatial coordinates)
 f = intensity (gray level, brightness)

digital image : x, y whole numbers

AND f a whole number

each element (x, y, f) is called a pixel
 (picture element)

THE TWO STEPS, NAMED

Sampling — making the *coordinates* discrete. It fixes how many pixels there are.

Quantisation — making the *amplitude* discrete. It fixes how many gray levels each pixel may take. Do not swap these two round; it is the most common slip in this unit.

1.2 The two resolutions

Spatial resolution is set by the sampling. In plain words it is the smallest detail you can still make out. Printers call the same thing *dots per inch*. **Intensity level resolution** is set by the quantisation: the number of gray levels, always quoted in bits.

$L = 2^k$ $L = \text{number of gray levels}$
 $k = \text{bits per pixel}$

$k : 1 \quad 2 \quad 4 \quad 6 \quad 8 \quad 16$
 $L : 2 \quad 4 \quad 16 \quad 64 \quad 256 \quad 65536$

pixel values are whole numbers in $[0, L-1]$

	SPATIAL RESOLUTION	INTENSITY LEVEL RESOLUTION
Set by	Sampling	Quantisation
Counts	How many pixels — $M \times N$	How many shades — $L = 2^k$
Cut it too far and you see	Blur first, then blocky squares — the checkerboard effect	Flat patches with hard edges — false contouring
Noticed first in	Fine texture and thin edges	Large smooth areas — sky, skin, a plain wall
In one word	Position	Brightness

THE TWO ARE NOT INDEPENDENT IN PRACTICE

A busy picture full of fine detail survives with **fewer** gray levels, because the texture hides the banding. A picture with large smooth areas needs **more** gray levels, because there is nothing to hide the false contours. So the right choice for one depends on the other and on the content.

1.3 Storage

$$b = M \times N \times k \quad b = \text{total bits}$$

$M = \text{rows}, N = \text{columns}$

$k = \text{bits per pixel}$

8 bits = 1 byte 1024 bytes = 1 KB 1024 KB = 1 MB

WORKED EXAMPLE

Three that cover every variant of the question.

(a) How many bits for a 1024×1024 image with 256 gray levels?

$$\begin{aligned}
 256 &= 2^8 \quad \text{so} \quad k = 8 \\
 b &= 1024 \times 1024 \times 8 = 8,388,608 \text{ bits} \\
 &= \frac{8,388,608}{8} = 1,048,576 \text{ bytes} \\
 &= \frac{1,048,576}{1024} = 1024 \text{ KB} = 1 \text{ MB exactly}
 \end{aligned}$$

(b) A 512×512 image with 32 gray levels.

$$\begin{aligned}
 32 &= 2^5 \quad \text{so} \quad k = 5 \\
 b &= 512 \times 512 \times 5 = 1,310,720 \text{ bits} \\
 &= 163,840 \text{ bytes} = 160 \text{ KB}
 \end{aligned}$$

(c) Going from 1024×1024 down to 256×256 — what fraction of the pixels is thrown away?

$$\begin{aligned}
 1024 \times 1024 &= 1,048,576 \text{ pixels} \\
 256 \times 256 &= 65,536 \text{ pixels} \\
 \text{lost} &= 983,040 \\
 983,040 / 1,048,576 \times 100 &= 93.75 \%
 \end{aligned}$$

The side is divided by 4, but the *count* is divided by 16. That is the whole trick in this question, and it is why halving the resolution of a photograph looks so harmless while the file shrinks so much.

EXAM TIP

Read *L* and convert it to *k* before you touch the multiplication. Nearly every lost mark here is a student who used 256 as the bits per pixel instead of 8.

CHAPTER 01 AT A GLANCE

- Sampling makes the **coordinates** discrete; quantisation makes the **amplitude** discrete. Sampling fixes pixel count, quantisation fixes shade count.
- $L = 2^k$ gray levels, values in $[0, L-1]$; storage is $b = M \times N \times k$ bits, and 8 bits = 1 byte.
- Too few samples → blur, then the **checkerboard effect**. Too few levels → **false contouring**, seen first in smooth areas.
- Halving each side quarters the pixel count — 1024 to 256 loses 93.75 % of the pixels.
- Detailed pictures survive on fewer gray levels; smooth pictures need more.

CHAPTER 02

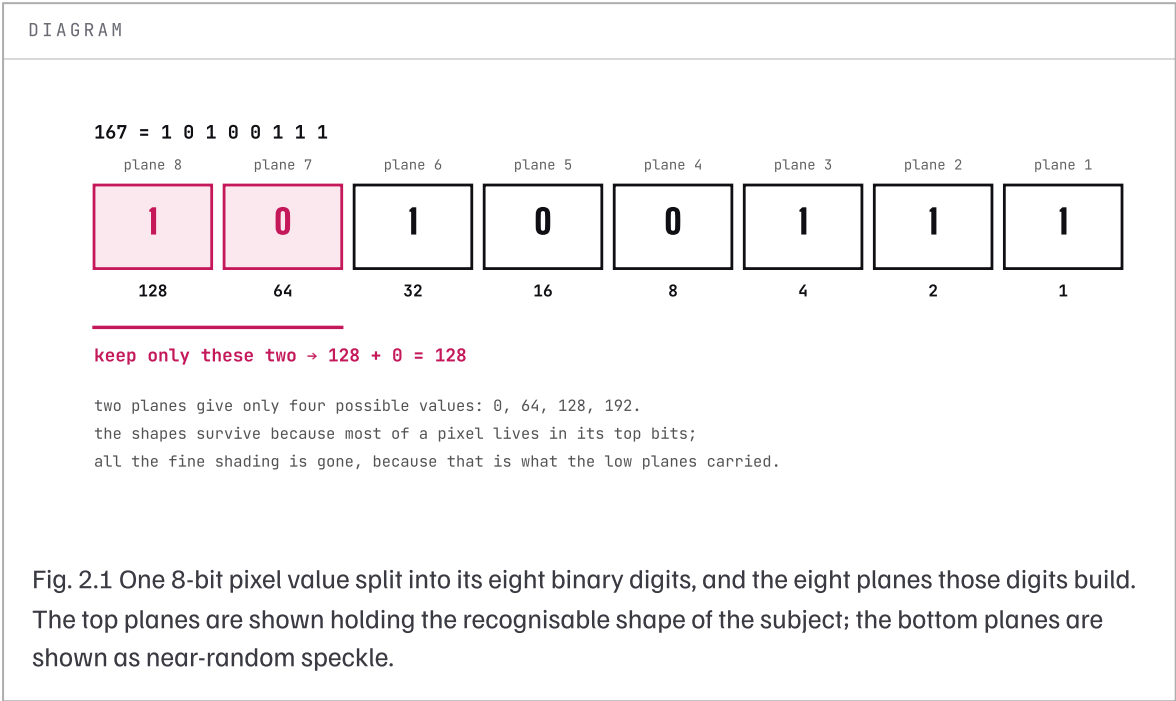
BIT PLANES

An 8-bit image is really eight stacked binary images. This chapter settles what each plane holds, how to rebuild an image from a few of them, and how to do the arithmetic in ninety seconds. It has appeared in both papers, always as a three-mark question with two pixel values in bold.

2.1 What a bit plane is

Write every pixel of an 8-bit image in binary. Now collect all the bit-1s into one image, all the bit-2s into another, and so on. You get eight binary images, each the size of the original. That is **bit-plane slicing**. It is not a new picture, only a different way of stacking the same bits.

bit	:	8	7	6	5	4	3	2	1
weight	:	128	64	32	16	8	4	2	1
		^						^	
		MSB				LSB			
		plane 8 = the				plane 1 = the			
		most significant				least significant			



	HIGHER-ORDER PLANES (8, 7, 6)	LOWER-ORDER PLANES (3, 2, 1)
Carry	The visually significant data	The subtle shading detail
Look like	A recognisable, coarse version of the picture	Near-random speckle
Weight of one bit	128, 64, 32 — large	4, 2, 1 — tiny
Used for	Compression, coarse thresholding, watermark carriers	Hiding data; they are the first thing a compressor throws away

2.2 Reconstruction from a few planes

COMPUTED · TOOLS/MAKE-FIGURES.PY

The top planes carry the picture; the bottom ones carry noise. Two planes give four grey levels: 0, 64, 128, 192

the image, 8 bits

plane 8 (128)

plane 7 (64)

plane 6 (32)

plane 5 (16)

planes 8 + 7 only

plane 4 (8)

plane 3 (4)

plane 2 (2)

plane 1 (1)

All eight planes of a real photograph. The top three are recognisable; planes 1 to 4 are very nearly noise. Keeping only planes 8 and 7 leaves four possible values — 0, 64, 128, 192 — which is exactly why the shapes survive and the shading does not.

THE WHOLE METHOD, IN ONE LINE

Write the pixel in 8-bit binary → keep only the bits belonging to the planes you were told to keep → set every other bit to zero → add up the weights that survive.

19 SEPTEMBER 2026

7 / 80

FROM THE PAPER

$$A = \begin{bmatrix} \mathbf{167} & 133 & \mathbf{111} \\ 144 & 140 & 135 \\ 159 & 154 & 148 \end{bmatrix}$$

Fig.3

Fig. 2.2 The 3×3 image A from the September 2025 paper. Only the two pixels in bold, 167 and 111, are wanted — the rest of the grid is there to make you find them.

WORKED EXAMPLE

Combine the 8th and 7th bit planes for the pixels 167 and 111.

bit :	8	7	6	5	4	3	2	1
weight :	128	64	32	16	8	4	2	1
167 =	1	0	1	0	0	1	1	1
check :	128 + 32 + 4 + 2 + 1 = 167 OK							
111 =	0	1	1	0	1	1	1	1
check :	64 + 32 + 8 + 4 + 2 + 1 = 111 OK							

Keep planes 8 and 7, zero the rest:

167 :	bit8 = 1	→ 128	,	bit7 = 0	→ 0	new value = 128
111 :	bit8 = 0	→ 0	,	bit7 = 1	→ 64	new value = 64

167 → 128 · 111 → 64

The same question, 2026 wording, with 200 and 75:

200 =	1	1	0	0	1	0	0	0	check	128+64+8 = 200	→	128 + 64 = 192
75 =	0	1	0	0	1	0	1	1	check	64+8+2+1 = 75	→	0 + 64 = 64

200 → 192 · 75 → 64

2.3 How many values can survive

Two planes carry two bits, so only four different outputs are possible. Saying this out loud is worth the last mark every time.

keep planes 8 and 7 only:

bit8	bit7	value
0	0	0
0	1	64
1	0	128
1	1	192

n planes kept $\rightarrow 2^n$ distinct output values

So the reconstruction is a coarse four-level version of the picture. The large shapes survive, because most of a pixel value lives in its top bits. Every trace of fine shading is gone, because that is what the lower planes carried.

THE NUMBERING TRAP

Here the **8th** plane means the MSB, counting from 1 at the LSB. Many books number the planes 0 to 7 instead, and then the same two planes are called the 7th and the 6th. Write one line saying which convention you are using. It takes a second and it removes the ambiguity.

EXAM TIP

Always write the check line — $128 + 32 + 4 + 2 + 1 = 167$ — under your binary. It costs five seconds, it catches a mis-copied bit, and examiners give marks for it.

CHAPTER 02 AT A GLANCE

- Bit-plane slicing splits an 8-bit image into **eight binary images**; plane 8 is the MSB with weight 128.
- Higher planes hold the visually significant shape; lower planes hold fine detail and look like speckle.
- To reconstruct: write the binary, keep the wanted bits, zero the rest, add the surviving weights.
- Keeping planes 8 and 7 gives only **four** possible outputs — 0, 64, 128, 192; keeping n planes gives 2^n values.
- Say which numbering convention you use — the MSB is the “8th” plane here, the “7th” in books that count from 0.

CHAPTER 03

NEIGHBOURS, ADJACENCY, PATHS AND DISTANCE

This is the four-mark question in both papers, and it is entirely mechanical once you have the definitions. The chapter settles the three neighbourhoods, the three adjacencies, why m-adjacency was invented, and the three distance formulas — plus the one trap where a formula gives a number but no path exists at all.

3.1 The three neighbourhoods

Take a pixel p at (x, y) . Three sets are defined, and they depend only on **position**, never on pixel values.

$$N_4(p) = (x-1, y)(x+1, y)(x, y-1)(x, y+1)$$

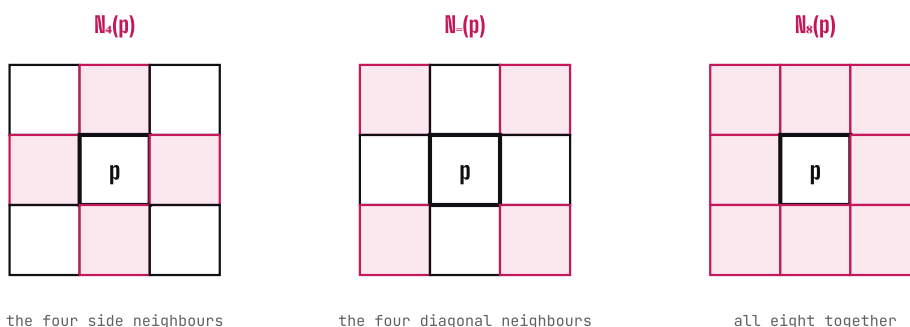
the 4 side neighbours

$$N_D(p) = (x-1, y-1)(x-1, y+1)(x+1, y-1)(x+1, y+1)$$

the 4 corner neighbours

$$N_8(p) = N_4(p) \cup N_D(p) \quad \text{all 8}$$

DIAGRAM



a border pixel has fewer — the neighbours that fall off the image simply do not exist

Fig. 3.1 The three neighbourhoods drawn on a 3×3 grid with p at the centre: N_4 marked on the four sides, N_D on the four corners, N_8 on all eight, with the coordinate of each cell written in.

THE SENTENCE EXAMINERS LOOK FOR

If p is a **border pixel**, some of these neighbours fall outside the image, so it has fewer than eight. Write that line whenever the pixel is on an edge — it carries a mark of its own.

3.2 The set V , and the three adjacencies

Two pixels sitting next to each other are *not* automatically adjacent. First fix V , the set of intensity values you are calling “the object”. In a binary image $V = \{1\}$; in a gray image V can be any subset, such as $V = \{0, 3\}$. Both pixels must have values in V and be positioned close enough.

ADJACENCY	CONDITION	IN WORDS
4-adjacency	q is in $N_4(p)$	They share a full side
8-adjacency	q is in $N_8(p)$	They share a side or just a corner
m-adjacency (mixed)	q is in $N_4(p)$, or q is in $N_D(p)$ and $N_4(p) \cap N_4(q)$ holds no pixel with a value in V	A side touch always counts; a corner touch counts only if there is no way round through a shared side-neighbour that is also in V

3.3 Why m-adjacency exists

8-adjacency can connect two pixels in more than one way at once, so “the path between p and q ” is not a single definite thing. m-adjacency removes the ambiguity by refusing a diagonal step whenever a route round it already exists.

WORKED EXAMPLE

For the image below with $V = \{1\}$, classify every touching pair. Positions are written (row, column) from 0.

```

      col 0  1  2
row 0 :   0  1  1
row 1 :   0  1  0
row 2 :   0  0  1

```

object pixels : (0,1) (0,2) (1,1) (2,2)

PAIR	TOUCH	4-ADJ	8-ADJ	M-ADJ
(0,1) – (0,2)	side	yes	yes	yes
(0,1) – (1,1)	side	yes	yes	yes
(0,2) – (1,1)	corner	no	yes	no
(1,1) – (2,2)	corner	no	yes	yes

The two corner pairs behave differently, and the working is the mark:

```

(0, 2)  and  (1, 1):   $N_4(0, 2) \cap N_4(1, 1) = \{(0, 1), (1, 2)\}$ 
                     $(0, 1) = 1$  IS in  $V \rightarrow$  a way round exists
                     $\rightarrow$  corner step REFUSED, not m-adjacent

(1, 1)  and  (2, 2):   $N_4(1, 1) \cap N_4(2, 2) = \{(1, 2), (2, 1)\}$ 
                     $(1, 2) = 0, (2, 1) = 0$  neither in  $V$ 
                     $\rightarrow$  corner step ALLOWED, m-adjacent

```

Counting the whole image: **2** four-adjacent pairs, **4** eight-adjacent pairs, **3** m-adjacent pairs.

EXAM TIP

Always write the intersection set out in full — $N_4(p) \cap N_4(q) = \{(1, 2), (2, 1)\}$, both are 0, none in V , therefore m-adjacent. The marks are on that line, not on the yes or no.

3.4 Paths, connectivity and regions

- A **path** from **p** to **q** is a sequence of pixels in which each one is adjacent to the next. Its **length** is the number of *steps*, which is one less than the number of pixels.

- p and q are **connected** in a set S if a path between them lies entirely inside S . The set of everything connected to p is its **connected component**.
- A set with exactly one connected component is a **region**. The **boundary** of a region is the pixels of the region that have at least one neighbour not in it.

3.5 The three distances

A function D is a distance (a metric) only if all three hold: $D(p, q) \geq 0$ and zero only when $p = q$; $D(p, q) = D(q, p)$; and $D(p, z) \leq D(p, q) + D(q, z)$.

$$p = (x, y) \quad q = (s, t)$$

$$\text{Euclidean : } D_e(p, q) = \sqrt{(x - s)^2 + (y - t)^2}$$

$$\text{City-block : } D_4(p, q) = |x - s| + |y - t|$$

$$\text{Chessboard : } D_8(p, q) = \max(|x - s|, |y - t|)$$

$$\text{always : } D_8 \leq D_e \leq D_4$$

DIAGRAM

D_4 - CITY BLOCK

4	3	2	3	4
3	2	1	2	3
2	1	0	1	2
3	2	1	2	3
4	3	2	3	4

D_8 - CHESSBOARD

2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

EUCLIDEAN

2.8	2.2	2.0	2.2	2.8
2.2	1.4	1.0	1.4	2.2
2.0	1.0	0.0	1.0	2.0
2.2	1.4	1.0	1.4	2.2
2.8	2.2	2.0	2.2	2.8

$D_4 = |x-s| + |y-t|$ – contours are diamonds. The pixels at $D_4 \leq 1$ are exactly $N_4(p)$.

$D_8 = \max(|x-s|, |y-t|)$ – contours are squares. The pixels at $D_8 \leq 1$ are exactly $N_8(p)$.

Euclidean is the true distance, but it is not an integer, which is why the other two exist.

Fig. 3.3 Contours of constant distance from a centre pixel: D_4 forms a diamond, D_8 forms a square, with the ring of value 1 and the ring of value 2 labelled on each. Draw both grids from memory — they answer half the questions on this topic.

	D_4 CITY-BLOCK	D_8 CHESSBOARD	D_M
Moves allowed	Horizontal and vertical only	Diagonals too, same cost	Along the shortest m-path
Shape of equal distance	Diamond	Square	No fixed shape
Pixels at distance 1	Exactly $N_4(p)$	Exactly $N_8(p)$	Depends on V
Depends on pixel values?	No — coordinates only	No — coordinates only	Yes — and on the neighbours of the path too

THE TRAP THE EXAMINER LOVES

D_4 and D_8 are formulas on coordinates. They are answered without looking at the picture. D_M is a **path length in the picture**, so the same two pixels can give four different answers depending on what lies between them. Worse, the formula can hand you a number when no path exists at all.

3.6 The full worked question

FROM THE PAPER

0 (0,0)	1	1	2	3 (q)
1	1	2	0	1
2	2	3	3	2
1	3	0	0 (p)	1
0	0	2	0	1

Fig.5

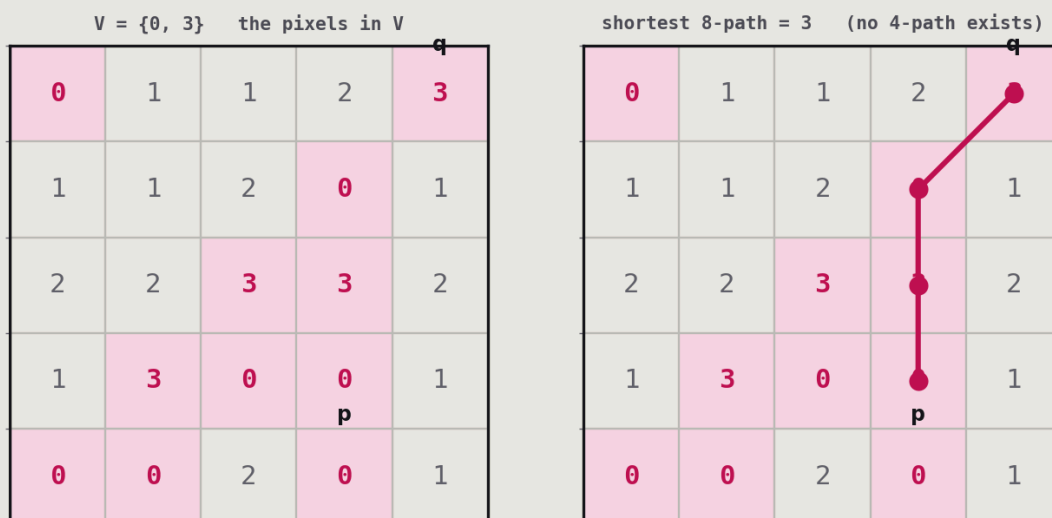
Fig. 3.4 The 5×5 image segment from the September 2025 paper, with **p** in the bottom row and **q** in the top-right corner. Every path question in this course looks like this one.

WORKED EXAMPLE

The segment, written with (row, column) from 0.

COMPUTED · TOOLS/MAKE-FIGURES.PY

q's only 4-neighbours are 2 and 1, neither in V — so under 4-adjacency q cannot be reached at all.



The 5×5 segment from the September 2025 paper with $V = \{0, 3\}$. The pixels in V are tinted; the shortest 8-path from p to q is drawn on the right. A breadth-first search over the same grid confirms it: **no 4-path exists at all**, because q 's only 4-neighbours are 2 and 1 and neither is in V .

(i) Neighbourhoods of $p = (3,3)$. Positions only — V does not enter here.

$$N_4(p) = (2,3)(4,3)(3,2)(3,4)$$

$$N_D(p) = (2,2)(2,4)(4,2)(4,4)$$

$$N_8(p) = (2,2)(2,3)(2,4)(3,2)(3,4)(4,2)(4,3)(4,4)$$

If the question then asks which of them lie in $V = \{0, 2, 3\}$, check the values:

$$(2,3) = 3 \quad \text{in} \quad (4,3) = 0 \quad \text{in} \quad (3,2) = 0 \quad \text{in} \quad (3,4) = 1 \quad \text{NOT}$$

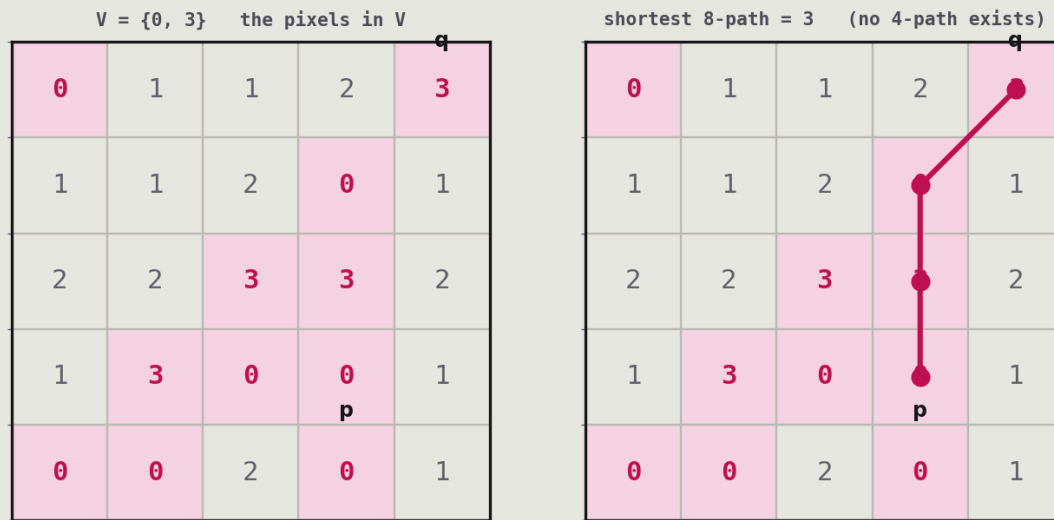
$$(2,2) = 3 \quad \text{in} \quad (2,4) = 2 \quad \text{in} \quad (4,2) = 2 \quad \text{in} \quad (4,4) = 1 \quad \text{NOT}$$

seven of the eight qualify

(ii) Shortest paths from $p = (3,3)$ to $q = (0,4)$, with $V = \{0, 3\}$. Mark the pixels that are in V first — this one step prevents most errors:

COMPUTED · TOOLS/MAKE-FIGURES.PY

q's only 4-neighbours are 2 and 1, neither in V — so under 4-adjacency q cannot be reached at all.



The paper's own grid with $V = \{0, 3\}$, and the shortest 8-path drawn. A breadth-first search over the same array confirms it, and confirms that no 4-path exists at all.

Shortest 4-path: none exists. Look at q and stop:

the 4-neighbours of $q = (0, 4)$ inside the image are

$(0, 3) = 2 \rightarrow \text{NOT in } V$

$(1, 4) = 1 \rightarrow \text{NOT in } V$

Neither is in V , so q is completely isolated under 4-adjacency. Nothing can reach it, however long the route. Note that the formula still returns $D_4 = |3 - 0| + |3 - 4| = 4$ — a number for a path that does not exist. That is exactly the trap.

Shortest 8-path = 3. Diagonals now count, and q does have a diagonal neighbour in V :

$(3, 3) \rightarrow (2, 3) \rightarrow (1, 3) \rightarrow (0, 4)$

p vertical vertical diagonal

values : $0 \rightarrow 3 \rightarrow 0 \rightarrow 3$ all in V

length = 3 steps

lower bound : $D_8 = \max(|3 - 0|, |3 - 4|) = \max(3, 1) = 3$ so 3 is optimal

Shortest m-path = 3. Test the one diagonal step:

$(3,3) \rightarrow (2,3)$ and $(2,3) \rightarrow (1,3)$ are 4-adjacent, allowed at once.

step $(1,3) \rightarrow (0,4)$ is diagonal:

$N_4(1,3) = \{ (0,3), (2,3), (1,2), (1,4) \}$

$N_4(0,4) = \{ (0,3), (1,4) \}$

common = $\{ (0,3), (1,4) \}$

values : $(0,3) = 2$ not in V , $(1,4) = 1$ not in V

nothing in $V \rightarrow$ the diagonal IS allowed

length = 3 steps

4-path: none · 8-path: 3 · m-path: 3. Finish with the comparison: m-adjacency only ever *removes* a diagonal link from 8-adjacency, and only when a route round already exists. So an m-path is never shorter than the 8-path, and is often the same length. What it buys is that the path is **unique**, which is the whole reason it was invented.

CHAPTER 03 AT A GLANCE

- N_4 = four sides, N_D = four corners, N_8 = all eight; a border pixel has fewer than eight.
- Adjacency needs **both** values in V and the right position; m-adjacency allows a diagonal only if $N_4(p) \cap N_4(q)$ holds nothing from V .
- m-adjacency exists to make the path **unique**; it never makes a path shorter than the 8-path.
- $D_4 = |x - s| + |y - t|$, $D_8 = \max(|x - s|, |y - t|)$, $D_e = \sqrt{(x-s)^2 + (y-t)^2}$, and always $D_8 \leq D_e \leq D_4$.
- D_4 and D_8 use coordinates only; D_m is a real path length, so it changes when the pixels in between change.
- A distance formula can return a number even when no path of that type exists — check the neighbours of q before you start walking.

CHAPTER 04

2-D SIGNALS AND CONVOLUTION

Every filter in this course is a convolution, so this is the machinery chapter. It settles the 2-D impulse, the size rule that must be written before any arithmetic, the address-sum method that does the whole calculation, and the two free checks that tell you whether the answer is right.

4.1 The 2-D impulse

Written as a signal, an image is $x(n_1, n_2)$ — the same table of numbers, new notation. The unit impulse is a single 1 at the origin.

```
delta(n1, n2) = 1    if n1 = 0 and n2 = 0
delta(n1, n2) = 0    otherwise
```


0 0 0 0 0	0 0 1 0 0	1 0 0 0 0
0 0 0 0 0	0 0 1 0 0	0 1 0 0 0
0 0 1 0 0	0 0 1 0 0	0 0 1 0 0
0 0 0 0 0	0 0 1 0 0	0 0 0 1 0
0 0 0 0 0	0 0 1 0 0	0 0 0 0 1

delta(n1,n2)	line impulse	diagonal impulse
the point	delta(n1)	delta(n1 - n2)

It matters because **any** sequence can be written as a sum of shifted, scaled impulses. That single fact is what makes convolution work.

4.2 The size rule — write this first

```
y = x ** h
```



```
if x is M1 x M2    and    h is N1 x N2
then y is (M1 + N1 - 1) x (M2 + N2 - 1)
```


ADD the sizes and subtract one, in each direction separately.
NEVER multiply them.

WHY IT GOES FIRST

The size tells you how many numbers you are looking for. Half the marks in a numerical convolution come from getting the size right, and a wrong size means every entry after it is in the wrong box.

4.3 The address-sum method

The whole method is one rule: a value of x at address (a, b) times a value of h at address (c, d) lands in y at address $(a + c, b + d)$. Place every product, then add whatever landed in each box.

$$y(n_1, n_2) = \sum_{k_1} \sum_{k_2} x(k_1, k_2) \cdot h(n_1 - k_1, n_2 - k_2)$$

in practice : $x(a, b) \cdot h(c, d)$ goes to $y(a + c, b + d)$

WORKED EXAMPLE

x is 2×3 , h is a column of three 1s.

$$\begin{array}{rcc} x = & 4 & 5 & 6 \\ & 7 & 8 & 9 \end{array} \qquad \begin{array}{r} h = \\ 1 \\ 1 \\ 1 \end{array}$$

Step 1 — the size. x is 2×3 , h is 3×1 , so y is $(2+3-1) \times (3+1-1) = 4 \times 3$.

Step 2 — the addresses.

$$\begin{array}{lll} x : (0,0) = 4 & (0,1) = 5 & (0,2) = 6 \\ (1,0) = 7 & (1,1) = 8 & (1,2) = 9 \\ (2,0) = 1 & & \end{array} \quad \begin{array}{l} h : (0,0) = 1 \\ (1,0) = 1 \\ (2,0) = 1 \end{array}$$

Step 3 — place and collect. For example the box (1,0) receives $x(0,0) \cdot h(1,0) = 4$ and $x(1,0) \cdot h(0,0) = 7$, so $y(1,0) = 4 + 7 = 11$.

$$\begin{array}{rcc} y = & 4 & 5 & 6 \\ & 11 & 13 & 15 \\ & 11 & 13 & 15 \\ & 7 & 8 & 9 \end{array} \qquad 4 \times 3$$

Read it and it makes sense: h is a column of three 1s, so the convolution adds up every group of three vertically neighbouring pixels. The first and last rows are the original rows on their own, with the kernel hanging half outside.

The two free checks.

$$\begin{aligned} \text{corner : } y(0,0) &= x(0,0) \cdot h(0,0) = 4 \times 1 = 4 \text{ OK} \\ \text{total : } \sum(y) &= \sum(x) \times \sum(h) \\ \sum(x) &= 4 + 5 + 6 + 7 + 8 + 9 = 39, \sum(h) = 3 \\ 39 \times 3 &= 117 \\ \sum(y) &= 15 + 39 + 39 + 24 = 117 \text{ OK} \end{aligned}$$

EXAM TIP

Do both checks, every time. The corner check catches a shifted answer; the total-sum check catches a dropped or doubled term. Together they take fifteen seconds and they are almost conclusive.

A SECOND ONE, 3×3 WITH 3×2

$$\begin{array}{ccc}
 x = & 1 & 2 & 3 \\
 & 4 & 5 & 6 \\
 & 7 & 8 & 9
 \end{array}
 \quad
 \begin{array}{cc}
 h = & 1 & 1 \\
 & 1 & 1 \\
 & 1 & 1
 \end{array}$$

$$\text{size : } (3+3-1) \times (3+2-1) = 5 \times 4$$

$$\begin{array}{cccc}
 y = & 1 & 3 & 5 & 3 \\
 & 5 & 12 & 16 & 9 \\
 & 12 & 27 & 33 & 18 \\
 & 11 & 24 & 28 & 15 \\
 & 7 & 15 & 17 & 9
 \end{array}$$

Spot-check two of them so you trust the method:

$$y(0,0) = x(0,0) \cdot h(0,0) = 1 \times 1 = 1$$

only one product can land in a corner

$$\begin{aligned}
 y(1,1) &= x(0,0)h(1,1) + x(0,1)h(1,0) + x(1,0)h(0,1) + x(1,1)h(0,0) \\
 &= 1 + 2 + 4 + 5 = 12
 \end{aligned}$$

$$\begin{aligned}
 \text{total check : } \sum(x) &= 45, \sum(h) = 6, 45 \times 6 = 270 \\
 \sum(y) &= 12 + 42 + 90 + 78 + 48 = 270 \text{ OK}
 \end{aligned}$$

4.4 Mask convolution and zero padding

Filtering an image keeps the answer the *same size* as the input, so the mask must be given something to hang over at the border. The usual choice is **zero padding**: surround the image with a border of zeros.

$$g(x,y) = \frac{\sum_s \sum_t w(s,t) \cdot f(x+s, y+t)}{\sum_s \sum_t w(s,t)}$$

the denominator is the sum of the mask coefficients

WORKED EXAMPLE

Convolve the 3×3 image below with a 3×3 averaging mask, using zero padding. Find the output at (0,0) and at (0,1).

image	mask (box filter)
7 9 11	1 1 1
10 50 8	1 1 1 $\times 1/9$
9 5 6	1 1 1

Pad first. Write the padded array out before touching the arithmetic — most lost marks here are pixels taken from the wrong place.

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 7 & 9 & 11 & 0 \\ 0 & 10 & 50 & 8 & 0 \\ 0 & 9 & 5 & 6 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

g(0,0) — the mask hangs off the top and the left, so five of the nine values are padded zeros:

$$\begin{array}{rcl} \text{window :} & 0 & 0 & 0 \\ & 0 & 7 & 9 \\ & 0 & 10 & 50 \\ g(0,0) & = & (0+0+0+0+7+9+0+10+50) / 9 = 76 / 9 = 8.44 \end{array}$$

g(0,1) — only the row above is missing, so three zeros come in:

$$\begin{array}{rcl} \text{window :} & 0 & 0 & 0 \\ & 7 & 9 & 11 \\ & 10 & 50 & 8 \\ g(0,1) & = & (0+0+0+7+9+11+10+50+8) / 9 = 95 / 9 = 10.56 \end{array}$$

$$g(0,0) = 8.44 \cdot g(0,1) = 10.56$$

The comment worth adding: zero padding pulls the border pixels down towards zero, so a zero-padded average always **darkens the edges** of the image. That is the price of the method, and it is why replicating or mirroring the border exists as an alternative.

CONVOLUTION VERSUS CORRELATION

True convolution **rotates the mask by 180°** before sliding it. Filtering with a symmetric mask — every mask in this course except the gradient pair — gives the same answer either way, so the distinction is usually invisible. Say one line about it if the question uses the word *convolution* and the mask is not symmetric.

CHAPTER 04 AT A GLANCE

- Size first: $M1 \times M2$ convolved with $N1 \times N2$ gives $(M1+N1-1) \times (M2+N2-1)$. Add and subtract one; never multiply.
- The whole method: $x(a, b) \cdot h(c, d)$ lands at $y(a + c, b + d)$. Place every product, then add up each box.
- Two free checks — $y(0, 0) = x(0, 0)h(0, 0)$, and $\text{sum}(y) = \text{sum}(x) \times \text{sum}(h)$.
- Mask filtering keeps the image size, so the border needs **zero padding** — write the padded array out before calculating.
- Zero padding always darkens the edges; that is the cost of the simplest border rule.

CHAPTER 05

THE 2-D FOURIER TRANSFORM

Both papers open with this, and both want a formula and a sketch rather than a theory. The chapter settles three transform pairs — impulse, impulse pair, sinusoid — and the convolution theorem, which is what turns a hard picture question into one line of multiplication.

5.1 The transform pair

$$F(u, v) = \iint f(x, y) e^{-j2\pi(ux+vy)} dx dy$$

$$f(x, y) = \iint F(u, v) e^{+j2\pi(ux+vy)} du dv$$

u, v = spatial frequencies

$|F(u, v)|$ = the spectrum

what you are asked to sketch

Only two properties are needed to answer the questions that get set: the **sifting property** of the impulse, and Euler's identity $e^{jT} + e^{-jT} = 2 \cos T$.

5.2 The three pairs to memorise

THE ONE FACT BEHIND EVERY QUESTION OF THIS TYPE

An impulse **pair** transforms to a **cosine**, and a cosine transforms to an impulse pair.

The Fourier transform swaps the two, in either direction. A single impulse is the limiting case where the pair merges and the cosine flattens to a constant.

WORKED EXAMPLE

(i) $f(x,y) = \delta(x,y)$ — a single impulse at the origin.

$$\begin{aligned} F(u,v) &= \iint \delta(x,y) e^{-j2\pi(ux+vy)} dx dy \\ &= e^0 \text{ (sifting picks out } x = y = 0) \\ &= 1 \text{ for every } (u,v) \end{aligned}$$

$F(u,v) = 1$ — constant magnitude everywhere. The spectrum is a uniform white plane with no structure. A perfectly localised point in space contains *every* spatial frequency in equal measure.

(ii) $f(x,y) = 0.5[\delta(x-a, y) + \delta(x+a, y)]$ — two impulses on the x-axis, symmetric about the origin.

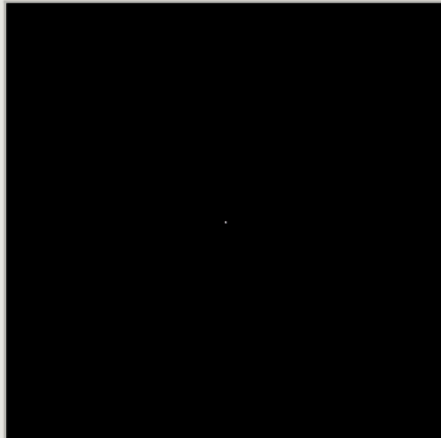
$$\begin{aligned} F(u,v) &= 0.5 [e^{-j2\pi u a} + e^{+j2\pi u a}] \\ \text{using } e^{jT} + e^{-jT} &= 2 \cos T : \\ F(u,v) &= 0.5 \times 2 \cos(2\pi u a) \\ &= \cos(2\pi u a) \quad \text{-- independent of } v \end{aligned}$$

$F(u,v) = \cos(2\pi u a)$. The spectrum $|F| = |\cos(2\pi u a)|$ is a set of **vertical bright and dark bands** parallel to the v-axis, spaced $1/(2a)$ apart along u.

COMPUTED · TOOLS/MAKE-FIGURES.PY

A point contains every frequency equally; a pair of points gives a cosine

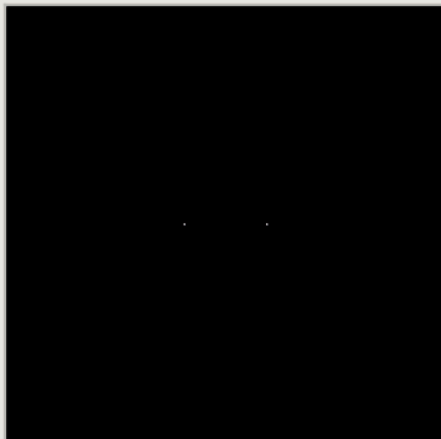
$\delta(x,y)$ one point



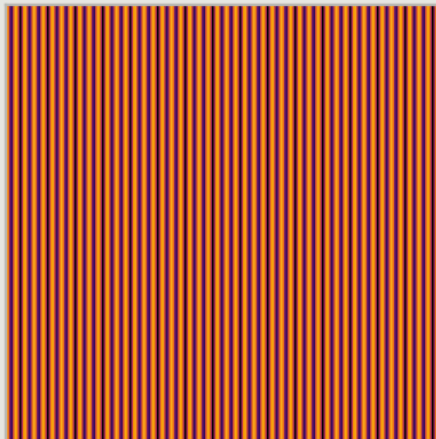
$|F| = 1$ everywhere a flat plane



$0.5[\delta(x-a,y) + \delta(x+a,y)]$



$|F| = |\cos(2\pi ua)|$ bands of period $1/(2a)$



Both pairs computed with `np.fft.fft2`. A single point gives a flat plane — every frequency in equal measure. A pair of points a apart gives $|\cos(2\pi ua)|$, which is bands of period $1/(2a)$: the further apart the points, the closer together the bands.

The reciprocal relationship, which is worth its own sentence: the further apart the impulses are in space (larger a), the more *closely* spaced the bands are in frequency. If the impulses are on the y -axis instead, swap u and v everywhere and the bands run horizontally.

5.3 A sinusoid gives an impulse pair

A sine wave that varies along `x` with 8 cycles across the image looks like **vertical stripes**. Its transform is a pair of impulses placed at its frequency, on the axis it varies along, plus a DC term at the origin because a real image is never negative.

sine along x , frequency 8 $\rightarrow$ impulses at $(u, v) = (+8, 0)$ and $(-8, 0)$
 sine along y , frequency 16 $\rightarrow$ impulses at $(u, v) = (0, +16)$ and $(0, -16)$
 plus DC at $(0, 0)$ in both cases

the higher the frequency, the FURTHER OUT the impulses sit

WHICH WAY THE STRIPES RUN

A sinusoid varying along x gives **vertical** stripes and puts its impulses on the **u-axis**.
 It is easy to draw the stripes the wrong way. Say “it varies along x , so the stripes are vertical, so the impulses are on the u -axis” out loud before you draw.

5.4 The convolution theorem

$$f1 ** f2 \quad \longleftrightarrow \quad F1(u, v) \times F2(u, v)$$

convolution in space = multiplication in frequency

multiplication in space = convolution in frequency

COMPUTED · TOOLS/MAKE-FIGURES.PY

The two images, generated from $\sin(2\pi u x/N)$

Fig.1 horizontal sine, frequency 8

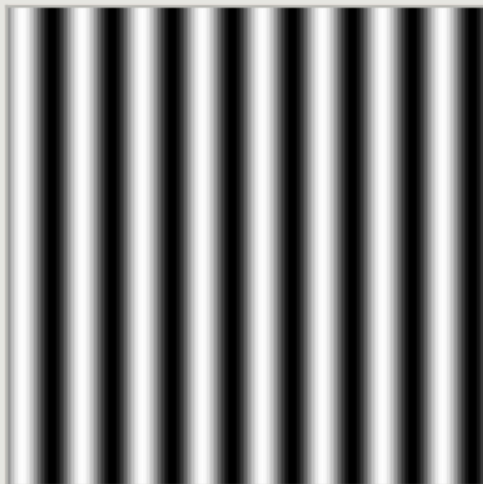


Fig.2 vertical sine, frequency 16



The two images the question is about, generated rather than drawn: Fig.1 is $0.5 + 0.5 \cdot \sin(2\pi \cdot 8x/N)$ and Fig.2 the same with 16 cycles down the columns. Fig.1 varies along x , so its stripes are **vertical**.

COMPUTED · TOOLS/MAKE-FIGURES.PY

A sinusoid transforms to a pair of impulses, on the axis it varies along.
The spectra are cropped to the middle 53×53 of 256×256 — everywhere else is exactly zero.

Fig.1 in space

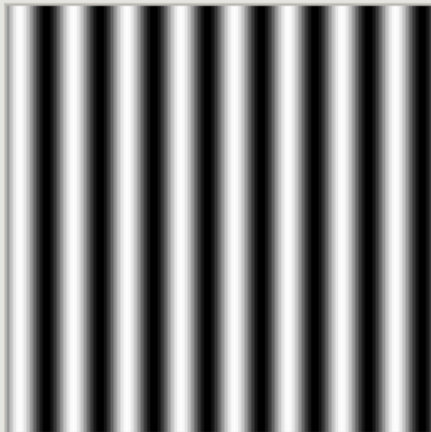
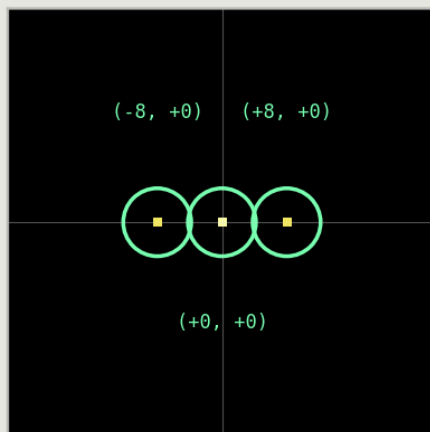
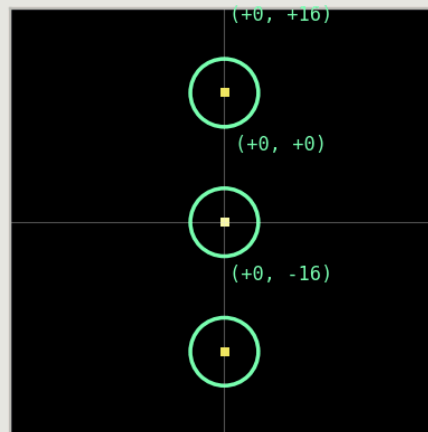


Fig.2 in space

|F1| on the u-axis, at ± 8 |F2| on the v-axis, at ± 16 

And their spectra, taken with `np.fft.fft2`. Three points each, and nothing anywhere else: Fig.1 at $(\pm 8, 0)$ on the u-axis, Fig.2 at $(0, \pm 16)$ on the v-axis, both with the DC term at the origin. The coordinates are read off the array, not asserted.

WORKED EXAMPLE

Fig.1 is a horizontal sine of frequency 8; Fig.2 is a vertical sine of frequency 16. Sketch the spectrum of their convolution.

Do not convolve the pictures. Use the theorem and look at where each spectrum is non-zero:

F_1 is non-zero only at $(0, 0), (+8, 0), (-8, 0)$	on the u-axis
F_2 is non-zero only at $(0, 0), (0, +16), (0, -16)$	on the v-axis
the product is non-zero only where BOTH are non-zero,	
and the only point they share is the ORIGIN.	

The result is a single impulse at the origin — the convolved image is a uniform grey field. Every stripe has vanished. The two DC terms multiply to give the constant; every other component is killed because the other image has nothing at that frequency to multiply it by.

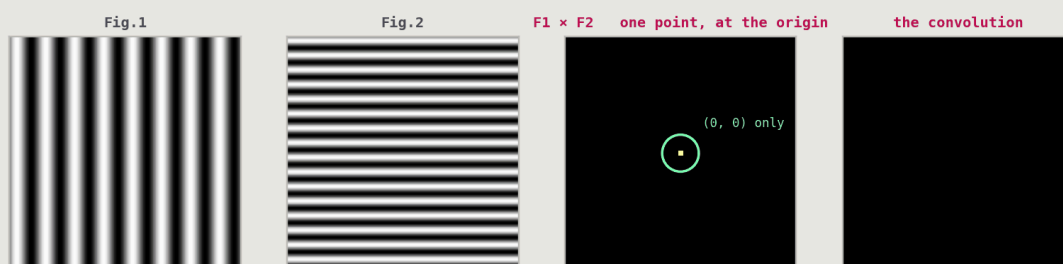
The cleaner way to say it: convolving with a pure sinusoid is a **filter that passes only that one frequency**. Fig.2 has no energy at $u = \pm 8$, so Fig.1's stripes go; Fig.1 has no energy at $v = \pm 16$, so Fig.2's stripes go. Nothing is left but the average brightness.

If the two sinusoids had been at the *same* frequency on the *same* axis, the impulses would coincide and the stripes would survive, amplified. Saying that earns the comment mark.

..

COMPUTED · TOOLS/MAKE-FIGURES.PY

The product is non-zero only where both spectra are — and the only point they share is the origin. Range of the result: 0



The whole argument, carried out. $F_1 \times F_2$ is non-zero at exactly one point — the origin, the only coordinate the two spectra share — and the inverse transform of that is a uniform field. The measured range of the result is printed on the figure, and it is effectively zero: every stripe really has gone.

IN SPACE	IN FREQUENCY
A single point (impulse)	A flat plane — all frequencies equally
Two points, $2a$ apart	Cosine bands of period $\frac{1}{2a}$
A sinusoid of frequency f	Two impulses at $\pm f$, plus DC
Convolution $f1 ** f2$	Multiplication $F1 \times F2$
Wide and smooth	Narrow, concentrated near the origin
Narrow and sharp	Wide and spread out
Sharp edges and fine detail	Energy far out from the origin

EXAM TIP

The log transform of Chapter 6 exists mainly to display these spectra. Values run from 0 to 10^6 , a screen has 256 levels, so without the log everything except the central spike shows as black.

CHAPTER 05 AT A GLANCE

- An impulse **pair** transforms to a **cosine**; a cosine transforms to an impulse pair. That one fact answers the standard question in either direction.
- $\delta(x, y) \rightarrow F(u, v) = 1$ — a point in space is flat in frequency.
- $0.5[\delta(x-a, y) + \delta(x+a, y)] \rightarrow \cos(2\pi ua)$ — bands of period $\frac{1}{2a}$; bigger a means tighter bands.
- A sinusoid varying along x gives vertical stripes and impulses on the **u-axis** at $\pm$ its frequency, plus DC.
- Convolution theorem: convolving two sinusoids at different frequencies leaves only the DC term — a uniform grey field.

CHAPTER 06

INTENSITY TRANSFORMATIONS

The simplest kind of enhancement: the new value of a pixel depends only on its own old value, so the whole operation is one curve $s = T(r)$. This chapter settles the three basic shapes, the direction gamma pushes the picture, and the piecewise family built from straight segments.

6.1 Point processing

$$s = T(r) \quad r = \text{input gray level}$$

s = output gray level

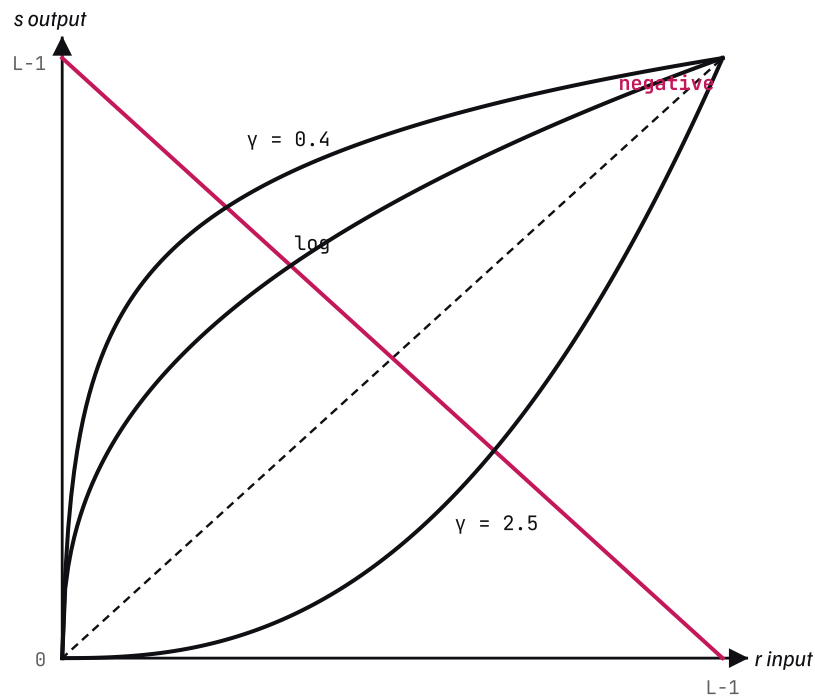
the neighbourhood is 1×1

only the pixel's own value is used

HOW TO READ ANY TRANSFORMATION GRAPH

A curve that bulges **above** the diagonal makes the picture **brighter**; a curve that sags **below** it makes the picture **darker**. The **steeper** the curve is over a range of inputs, the more **contrast** that range gets. Those two sentences answer most of the descriptive parts.

DIAGRAM



$\gamma < 1$ lifts the dark end, $\gamma > 1$ pushes it down.
anything above the identity line brightens; anything below darkens.

Fig. 6.1 All the basic transformations on one s against r square: the identity diagonal, the negative falling from top-left to bottom-right, the log and n -th root curves bulging above the diagonal, and the inverse log and n -th power curves sagging below it.

6.2 The negative

$$s = L - 1 - r \quad \text{for 8 bits: } s = 255 - r$$

$$r = 50 \rightarrow s = 205$$

$$r = 200 \rightarrow s = 55$$

Learn *when* it is used, because that is the usual question: the negative suits **white or gray detail embedded in dark regions**, especially when the black areas dominate in size. The classic case is a medical X-ray where the interesting tissue is bright but drowned in a large black surround.

6.3 The log transform

COMPUTED · TOOLS/MAKE-FIGURES.PY



The power law applied to a real image at $\gamma = 0.4, 1$ and 2.5 , with the three curves beside it. Anything above the diagonal brightens, anything below darkens — and you can see which is which without being told.

$$s = c \log(1 + r) \quad c \text{ a constant, } r \geq 0$$

the $+ 1$ is there so that $r = 0$ gives $s = 0$, not minus ∞

The log maps a **narrow range of low gray levels into a much wider range of outputs**, and does the opposite at the top end. In one sentence: it expands dark pixels and compresses bright ones. Its real job is **compressing dynamic range**. The **inverse log** does exactly the reverse.

WORKED EXAMPLE

Why a Fourier spectrum needs the log. Spectrum values run from 0 to about 1.5×10^6 . A display has 256 levels. Without the log, the whole image maps to level 0 except the single brightest spike.

with $c = 1$ and logs to base 10 :

$$r = 0 \quad \rightarrow \quad s = \log_{10}(1 + 0) = 0$$

$$r = 1.5 \times 10^6 \quad \rightarrow \quad s = \log_{10}(1 + 1500000) = 6.176$$

a range of 1,500,000 has been squeezed into a range of 6.2

Now scale that 0 to 6.2 across 0 to 255 and every part of the spectrum is visible at once. That is the standard example in the lecture, and it is the answer to “where is the log transformation actually used?”

6.4 Power law and gamma

$$s = cr^\gamma$$

c and γ both positive

$$\gamma < 1$$

curve ABOVE the diagonal expands dark values

image gets BRIGHTER

$$\gamma = 1$$

the diagonal itself identity no change

$$\gamma > 1$$

curve BELOW the diagonal compresses dark ones

image gets DARKER

VALUE OF γ	CURVE	EFFECT	USE IT WHEN THE IMAGE IS
$\gamma < 1$	Above the diagonal	Expands dark values, brightens	Too dark
$\gamma = 1$, c = 1	The diagonal	Identity, no change	—
$\gamma > 1$	Below the diagonal	Compresses dark values, darkens, more contrast at the top	Washed out

WORKED EXAMPLE

Gamma correction for a monitor. A display is not linear: the intensity at the screen is roughly the applied voltage raised to about 2.5, and that exponent is what the word *gamma* originally meant. Real devices sit between 1.8 and 2.5. Left alone, the picture comes out too dark.

The cure is to pre-distort by the **inverse** power before sending the image:

$$s = cr^{\frac{1}{\gamma}} \quad \text{for } \gamma = 2.5, \frac{1}{2.5} = 0.4$$

Put two pixels of an 8-bit image through it, normalising, applying, and scaling back:

$$\begin{aligned} \gamma = 0.4(\text{correcting}) \quad s &= 255 \times \left(\frac{r}{255}\right)^{0.4} \\ r = 50 &\rightarrow 255 \times (0.1961)^{0.4} = 132.9 \\ r = 128 &\rightarrow 255 \times (0.5020)^{0.4} = 193.6 \\ \gamma = 2.5(\text{the monitor itself}) \quad s &= 255 \times \left(\frac{r}{255}\right)^{2.5} \\ r = 50 &\rightarrow 4.3 \\ r = 200 &\rightarrow 138.9 \end{aligned}$$

Read the two columns together and the cancellation is visible: the correction lifts 50 to 132.9, the monitor then drops 132.9 back towards 50, and what you see matches what was captured.

EXAM TIP

Learn the direction by heart, because it is a one-mark question that is easy to get backwards. **Dark image** → γ **less than 1**. **Washed-out image** → γ **greater than 1**. And note that reducing γ too far stops helping: contrast falls and the background takes on a washed-out look of its own.

6.5 Piecewise linear transformations

The curves above are fixed shapes with one constant. A **piecewise linear** function is built from straight segments whose corners you place yourself, so it can take any shape at all. The price is that it needs more input from the user.

- **Contrast stretching** — two control points (r_1, s_1) and (r_2, s_2) decide the whole shape. The goal is to increase the dynamic range so the image spans the full display. Quote the three causes of low contrast: poor illumination, lack of dynamic range in the sensor, and the wrong lens aperture at acquisition.

- With $r_1 = r_2$ and $s_1 = 0, s_2 = L - 1$, the same function becomes a **thresholding** function — a binary image.
- **Intensity level slicing** — highlight one band of gray levels. Either set that band bright and everything else dark, or brighten the band and leave the rest unchanged.
- **Bit-plane slicing** is the last member of this family, and it has its own chapter.

CHAPTER 06 AT A GLANCE

- $s = T(r)$ — point processing, neighbourhood 1×1 . Above the diagonal brightens, below darkens, steeper means more contrast.
- Negative $s = L - 1 - r$; it suits **bright detail inside large dark regions**, such as an X-ray.
- $\text{Log } s = c \log(1 + r)$ expands dark, compresses bright; its real job is **dynamic-range compression**, as for a Fourier spectrum.
- Power law $s = c r^\gamma$: $\gamma < 1$ brightens a dark image, $\gamma > 1$ darkens a washed-out one. Correct a monitor of gamma 2.5 with $1/2.5 = 0.4$.
- Piecewise linear covers contrast stretching, thresholding and level slicing — any shape, but you must supply the corner points.

CHAPTER 07

THE HISTOGRAM, ITS STATISTICS AND LOCAL ENHANCEMENT

The histogram is the input to everything in the next two chapters, and two numbers taken from it — the mean and the variance — are enough to drive an adaptive filter. This chapter settles both, and then settles why a global transformation sometimes has to be replaced by a local one.

7.1 The histogram

$$h(r_k) = n_k \quad n_k = \text{how many pixels have gray level } r_k$$

$$\text{normalised : } p(r_k) = \frac{n_k}{MN} \quad \text{and} \quad \sum_k p(r_k) = 1$$

it is a count, NOT an image – all position information is lost

Two pictures with the same histogram can look completely different. That is the one limitation worth stating, and it is also why local processing exists at all.

7.2 Mean and variance, two ways

$$\text{mean} \quad m = \left(\frac{1}{MN} \right) \sum_x \sum_y f(x, y)$$

walk the image

$$m = \sum_i r_i p(r_i)$$

walk the histogram

$$\text{variance} \quad s_2 = \left(\frac{1}{MN} \right) \sum_x \sum_y [f(x, y) - m]^2$$

$$s_2 = \sum_i (r_i - m)^2 p(r_i)$$

mean measures BRIGHTNESS variance measures
CONTRAST

WORKED EXAMPLE

Compute the mean of this 2-bit 5×5 image by both formulas and compare.

0	0	1	1	2	$MN = 25$, values 0..3 (2 bits, $L = 4$)
1	2	3	0	1	
3	3	2	2	0	
2	3	1	0	0	
1	1	3	2	2	

Formula 1 — walk the image.

```

row sums : 0 + 0 + 1 + 1 + 2 = 4
           1 + 2 + 3 + 0 + 1 = 7
           3 + 3 + 2 + 2 + 0 = 10
           2 + 3 + 1 + 0 + 0 = 6
           1 + 1 + 3 + 2 + 2 = 9

           total = 36

            $m = \frac{36}{25} = 1.44$ 

```

Formula 2 — walk the histogram. Count each gray level first.

R	COUNT N	$P(R) = N/25$	$R \times P(R)$
0	6	0.24	0.00
1	7	0.28	0.28
2	7	0.28	0.56
3	5	0.20	0.60
total	25	1.00	1.44

Both give $m = 1.44$, and they must. The two are the same calculation organised differently. Formula 1 walks the image pixel by pixel; formula 2 groups identical pixels first and weights each distinct value by how often it occurs. Grouping cannot change a sum.

Why bother with the second, then? Because it costs one pass over the *histogram* instead of one pass over the *image*. For a 5×5 toy that is no saving; for a 4096×4096 image with 256 levels it is 256 terms instead of 16.7 million. And once the histogram exists, every other statistic is nearly free:

$$\begin{aligned}
 \text{variance} &= \sum (r_i - m)^2 p(r_i) \\
 &= (0 - 1.44)^2(0.24) + (1 - 1.44)^2(0.28) \\
 &\quad + (2 - 1.44)^2(0.28) + (3 - 1.44)^2(0.20) \\
 &= 0.4977 + 0.0542 + 0.0878 + 0.4867 \\
 &= 1.1264 \quad \text{so} \quad \sigma = 1.061
 \end{aligned}$$

STATISTIC	WHAT IT MEASURES	A HIGH VALUE MEANS
Mean m	Average intensity — the overall brightness	A brighter image; a low mean means a darker image
Variance σ^2 (or σ)	How spread out the intensities are — the contrast	Bigger differences between pixels, so more contrast; it can also mean a grainier, noisier picture

Variance earns its place because it is used everywhere: adaptive local contrast enhancement, Otsu thresholding (which compares within-class and between-class variance), noise estimation, and texture analysis — smooth regions have near-zero variance, sand and foliage have high variance.

7.3 Global is not always enough

Everything so far is **global**: one transformation, built from the gray levels of the *entire* image, applied to every pixel. That fails when the detail you want lives in a small area. A few hundred pixels cannot move a histogram built from a quarter of a million, so a locally dark region stays dark.

LOCAL HISTOGRAM PROCESSING — THE FIVE-STEP RECIPE

1. Define a neighbourhood, usually a square window.
2. Move its **centre** from pixel to pixel.
3. At each position compute the histogram of the pixels **inside that window only**, and get its equalisation transformation.
4. Use it to map **the centre pixel alone**.
5. Move on and repeat.

	SLIDING WINDOW	BLOCK (NON-OVERLAPPING)
One transformation per	Pixel	Block
Cost	High in principle, but the histogram is updated incrementally — one column leaves, one enters	Very low; one equalisation per block, and the blocks are independent so it parallelises trivially
Output quality	Smooth — neighbouring pixels have nearly identical windows, so nearly identical transformations	Visible blocking artefacts ; adjacent blocks get unrelated transformations, so a discontinuity appears at every boundary
Side effect	Small windows amplify noise into a finer texture	Seams
The usual fix	—	Equalise per block, then interpolate between neighbouring blocks' transformations — which is what CLAHE does

7.4 Adaptive local enhancement

Now the two statistics are put to work. The rule scans the image with a small 3×3 or 5×5 window and brightens **only the patches that are both dark and low in contrast**, leaving everything else untouched.

$$g(x, y) = C \cdot f(x, y) \quad \text{if} \quad m_S \leq k_0 \cdot mG$$

$$\text{AND } k_1 \cdot sG \leq s_S \leq k_2 \cdot sG$$

$$g(x, y) = f(x, y) \quad \text{otherwise}$$

mG, sG = global mean and standard deviation

m_S, s_S = local mean and standard deviation in the window

C = gain, $C > 1$ typical: $k_0 < 1, k_1 > k_0, k_2 < 1$

READING THE TWO CONDITIONS IN PLAIN WORDS

First condition $m_S \leq k_0 \cdot mG$ says *this patch is darker than the picture as a whole*, so it is a candidate for brightening. **Second condition** $k_1 \cdot sG \leq s_S \leq k_2 \cdot sG$ says *this patch has some variation but not much*. The lower bound throws away perfectly flat patches, which contain nothing to reveal; the upper bound throws away patches that already have plenty of contrast. Only what is left — dark **and** faint — gets multiplied by C .

EXAM TIP

If asked what role C plays: it is the **gain**, and it decides how strongly the qualifying pixels are boosted. Too small and nothing is revealed, too large and the patches saturate to white. The k values decide **which** pixels qualify; C decides **how much** they change.

CHAPTER 07 AT A GLANCE

- Histogram $h(r_k) = n_k$, normalised $p(r_k) = \frac{n_k}{MN}$ summing to 1; it keeps no position information.
- Mean two ways — $(1/MN)\sum\sum f(x,y)$ or $\sum r_i p(r_i)$ — and they must agree, because grouping cannot change a sum.
- Mean measures **brightness**, variance $\sum (r_i - m)^2 p(r_i)$ measures **contrast**.
- Global equalisation misses small regions; local equalisation builds a fresh transformation inside a window and maps **only the centre pixel**.
- Sliding window: one transformation per pixel, smooth, cheap because the histogram updates incrementally. Block method: fast but leaves **blocking artefacts**.
- Adaptive rule: multiply by C only where the patch is dark ($m_S \leq k_0 - m_G$) and faint ($k_1 - s_G \leq s_S \leq k_2 - s_G$).

CHAPTER 08

HISTOGRAM EQUALISATION

One formula, four steps, and a table. Equalisation spreads a crowded histogram over the full range and so improves contrast. This chapter settles the formula, the table you must draw, and the sentence about why the answer is never actually flat — which is where the last mark lives.

8.1 The idea

A low-contrast image has all its pixels crowded into a narrow band of gray levels. Equalisation stretches that band across the whole range. The transformation it uses is the **cumulative** histogram, scaled to $L-1$.

$$s_k = T(r_k) = (L - 1) \sum p_r(r_j)$$

$$j = 0 \dots k$$

$$= \frac{(L - 1) \times (\text{cumulative count up to } k)}{MN}$$

then $\text{ROUND } s_k$ to the nearest whole gray level

WHY THE CDF, AND NOT SOME OTHER CURVE

The transformation must be **single-valued and monotonically increasing**, so that the order of gray levels is preserved and the mapping can be undone; and it must map $[0, L-1]$ into $[0, L-1]$, so that the output fits the same display. The cumulative distribution satisfies both by construction, which is the whole reason it is chosen.

DIAGRAM

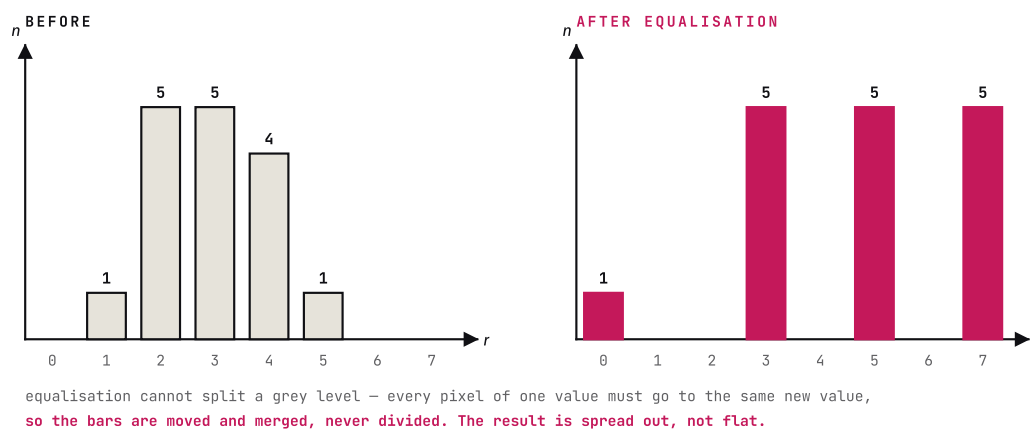


Fig. 8.1 Three panels: the crowded input histogram, the cumulative curve rising across it with dotted lines carrying each bar through the curve onto the output axis, and the resulting output histogram with the same bars now spread out and some of them merged.

8.2 The worked table

WORKED EXAMPLE

Equalise this 4 × 4 image with L = 8.

1332

4243

3235

2424

MN = 16 , L = 8 , L-1 = 7

Step 1 — count.

r_k : 01234567

n_k : 01554100

check : 0+1+5+5+4+1+0+0 = 16 OK

Step 2 — running sum and transformation. $s_k = 7 \times \text{cumulative} / 16$.

R	N	CUMULATIVE	S = 7×CUM/16	ROUNDED
0	0	0	0.0000	0
1	1	1	0.4375	0
2	5	6	2.6250	3
3	5	11	4.8125	5
4	4	15	6.5625	7
5	1	16	7.0000	7
6	0	16	7.0000	7
7	0	16	7.0000	7

Step 3 — the mapping and the output image.

1 → 0 2 → 3 3 → 5 4 → 7 5 → 7

input output

1 3 3 2 0 5 5 3

4 2 4 3 → 7 3 7 5

3 2 3 5 5 3 5 7

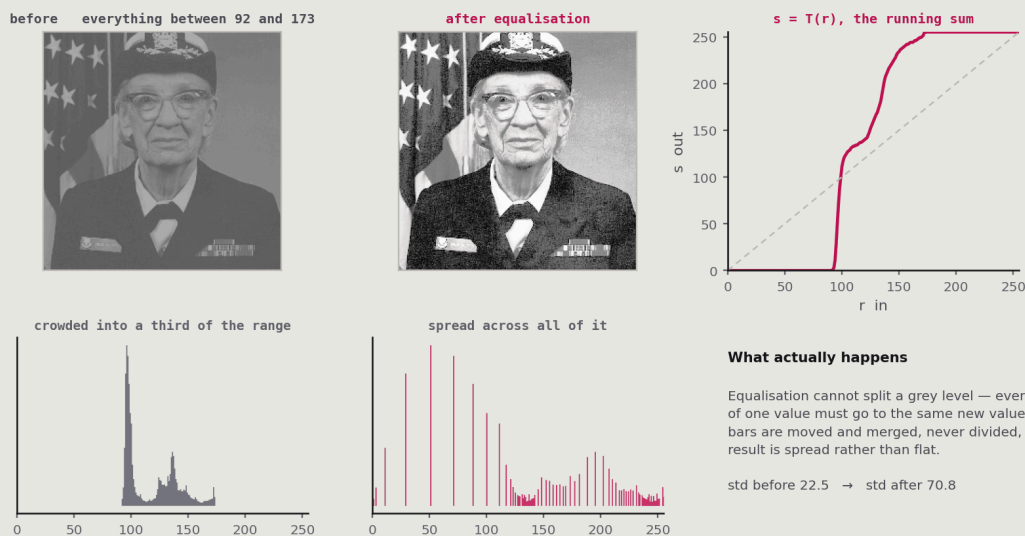
2 4 2 4 3 7 3 7

Step 4 — the output histogram.

```
s      : 0  1  2  3  4  5  6  7
count : 1  0  0  5  0  5  0  5
```

```
check : 1 + 5 + 5 + 5 = 16      OK
```

COMPUTED · TOOLS/MAKE-FIGURES.PY



A genuinely flat image, its histogram, the running sum that does the work, and the result. **The output histogram is not flat and never will be** — equalisation cannot split a grey level, so the bars are moved and merged, never divided. What it buys is the spread: standard deviation 22.5 before, 70.8 after.

8.3 Why the answer is never flat

THE COMMENT THAT EARNS THE LAST MARK

The output histogram is **not flat**, and in the discrete case it never will be. Equalisation cannot **split** a gray level — all five pixels of value 2 must go to the *same* new value — so the bars can only be **moved and merged**, never divided. What it does achieve is spreading them over the full range, and that is where the contrast improvement comes from. Only in the continuous case is the result exactly uniform.

EXAM TIP

Three free checks: the counts must sum to MN before and after; the last s_k must come out exactly $L-1$; and the mapping must be non-decreasing. If any of the three fails, the arithmetic is wrong.

Two more points worth one line each. Equalisation is **automatic** — it needs no parameter from the user, unlike a piecewise linear stretch. And equalising an already-equalised image changes nothing further, because the transformation is idempotent in the continuous case.

CHAPTER 08 AT A GLANCE

- $s_k = (L-1) \times (\text{cumulative count up to } k) / MN$, then round to the nearest gray level.
- The CDF is chosen because it is single-valued, monotonically increasing, and maps $[0, L-1]$ onto $[0, L-1]$.
- Draw the table: r_k, n_k , cumulative, s_k , rounded. Then the mapping, then the output image, then the output histogram.
- The result is **never flat** — bars can be moved and merged but never split, so only the continuous case gives a uniform histogram.
- Checks: counts sum to MN both times, and the last s_k must equal exactly $L-1$.

CHAPTER 09

HISTOGRAM MATCHING

Equalisation gives you one particular output shape and no say in it. Matching lets you ask for a shape. The method is equalisation done twice and then joined in the middle. This chapter settles the discrete table version and the continuous calculus version, because the papers ask for both.

9.1 What matching asks for

Also called **histogram specification**. Given an input image and a target histogram, find the transformation $r \rightarrow z$ that makes the input look as much like the target as it can.

equalise the input : $s = T(r)$

equalise the target : $s = G(z)$

both are now uniform, so they can be set
equal:

$$z = G^{-1}[T(r)]$$

THE THREE STEPS, AND THEY NEVER CHANGE

1. Equalise the input $\rightarrow T(r)$. **2.** Equalise the specified histogram $\rightarrow G(z)$. **3.** Set $G(z) = T(r)$ and solve for z . In the discrete case “solve” means *find the z whose $G(z)$ is nearest.*

9.2 The discrete version

WORKED EXAMPLE

A 3-bit image ($L = 8$, 45 pixels) has the histogram in Table I. Transform it to the specified histogram of Table II. Find the transformation and the actual output histogram.

TABLE I (input)									TABLE II (specified)								
r :	0	1	2	3	4	5	6	7	z :	0	1	2	3	4	5	6	7
n :	10	5	5	11	5	0	3	6	Pz :	5	5	5	10	10	5	5	0

total = 45 in both

Step 1 — equalise the input. $s = 7 \times \text{cumulative} / 45$.

R	N	CUM	$S = 7 \times \text{CUM} / 45$
0	10	10	1.5556
1	5	15	2.3333
2	5	20	3.1111
3	11	31	4.8222
4	5	36	5.6000
5	0	36	5.6000
6	3	39	6.0667
7	6	45	7.0000

Step 2 — equalise the specified histogram. $G = 7 \times \text{cumulative} / 45$.

Z	PZ	CUM	$G(Z)$
0	5	5	0.7778
1	5	10	1.5556
2	5	15	2.3333
3	10	25	3.8889
4	10	35	5.4444
5	5	40	6.2222
6	5	45	7.0000
7	0	45	7.0000

Step 3 — for each s , find the z whose $G(z)$ is closest.

R	S	NEAREST $G(z)$	→ Z
0	1.5556	1.5556 exact	1
1	2.3333	2.3333 exact	2
2	3.1111	tie — 2.3333 and 3.8889 are both 0.7778 away	2
3	4.8222	5.4444, gap 0.6222	4
4	5.6000	5.4444, gap 0.1556	4
5	5.6000	5.4444	4
6	6.0667	6.2222, gap 0.1556	5
7	7.0000	7.0000 exact	6

The tie at $r = 2$ matters. $s = 3.1111$ sits exactly halfway between $G(2)$ and $G(3)$. The convention is to **choose the smallest z** , so $z = 2$. This is why the question warns you not to round s to a whole number early — rounding 3.1111 to 3 first would have hidden the tie completely.

Step 4 — pour each count into its mapped z .

$$\begin{aligned}
 z = 1 &\leftarrow r_0 = 10 \\
 z = 2 &\leftarrow r_1 + r_2 = 5 + 5 = 10 \\
 z = 4 &\leftarrow r_3 + r_4 + r_5 = 11 + 5 + 0 = 16 \\
 z = 5 &\leftarrow r_6 = 3 \\
 z = 6 &\leftarrow r_7 = 6 \\
 z = 0, 3, 7 &= 0
 \end{aligned}$$

$$\text{total} = 10 + 10 + 16 + 3 + 6 = 45 \text{ OK}$$

z	0	1	2	3	4	5	6	7
specified	5	5	5	10	10	5	5	0
actual	0	10	10	0	16	3	6	0

The actual histogram **only approximates** the specified one, and it always will. Matching cannot split a gray level — all 11 pixels of value 3 must go to the same z — so the bars can only be moved and merged. In the continuous case the match is exact; in the discrete case you get the nearest achievable shape, and saying so is worth a mark.

9.3 The continuous version

When the question gives you two probability density functions instead of two tables, the same three steps become three integrals.

$$s = T(r) = (L - 1) \int p_r(w) dw \text{ from } 0 \text{ to } r$$

$$G(z) = (L - 1) \int p_z(t) dt \text{ from } 0 \text{ to } z$$

set $G(z) = T(r)$ and solve the algebra for z

WORKED EXAMPLE

An image has $p_r(r) = 2r/(L-1)^2$ for $0 \leq r \leq L-1$. Find the transformation giving $p_z(z) = 3z^2/(L-1)^3$.

Step 1

equalise the input

$$\begin{aligned} s = T(r) &= (L-1) \int_0^r \frac{2w}{(L-1)^2} dw \\ &= (L-1) \left[\frac{w^2}{(L-1)^2} \right]_0^r \\ &= \frac{r^2}{L-1} \end{aligned}$$

Step 2

equalise the target

$$\begin{aligned} G(z) &= (L-1) \int_0^z \frac{3t^2}{(L-1)^3} dt \\ &= (L-1) \left[\frac{t^3}{(L-1)^3} \right]_0^z \\ &= \frac{z^3}{(L-1)^2} \end{aligned}$$

Step 3

set them equal

$$\begin{aligned} \frac{z^3}{(L-1)^2} &= \frac{r^2}{L-1} \\ z^3 &= r^2(L-1) \\ z &= \text{cube root of } [r^2(L-1)] \end{aligned}$$

Verify at the two ends. A monotonic mapping of the full range must fix both endpoints:

$$\begin{aligned} r = 0: \quad z &= [0]^{\frac{1}{3}} = 0 \text{ OK} \\ r = L-1: \quad z &= [(L-1)^2(L-1)]^{\frac{1}{3}} \\ &= [(L-1)^3]^{\frac{1}{3}} = L-1 \text{ OK} \end{aligned}$$

A worked value for an 8-bit image, $L-1 = 255$:

$$\begin{aligned} r = 128 &\rightarrow z = [128^2 \times 255]^{\frac{1}{3}} \\ &= [16384 \times 255]^{\frac{1}{3}} \\ &= (4,177,920)^{\frac{1}{3}} \\ &= 161.1 \end{aligned}$$

Mid-grey is pushed **up** to 161. That is the right direction: the target grows as z^2 , so it wants more pixels at the bright end than the input's linear density supplies, and the transformation obliges by stretching values upwards.

A second one, where the algebra is a quadratic. Input density rising from 0 to $2/255$, target falling from $2/255$ to 0, both over $(0, 255)$:

$$\begin{aligned}
 p_F(f) &= 2f / 255^2 & p_G(g) &= 2(255-g) / 255^2 \\
 T(f) &= 255 \int_0^f 2w/255^2 dw & &= f^2 / 255 \\
 G(g) &= 255 \int_0^g 2(255-t)/255^2 dt & &= (510g - g^2) / 255 \\
 \text{set equal : } & 510g - g^2 = f^2 \\
 & g^2 - 510g + f^2 = 0 \\
 & g = [510 \pm \sqrt{510^2 - 4 f^2}] / 2 \\
 & = 255 \pm \sqrt{255^2 - f^2}
 \end{aligned}$$

Which root? The plus sign gives $g \geq 255$ for every f , which is outside the range. Take the minus sign:

$$\begin{aligned}
 g(f) &= 255 - \sqrt{65025 - f^2} \\
 f = 0 &\rightarrow g = 255 - 255 = 0 \text{ OK} \\
 f = 255 &\rightarrow g = 255 - 0 = 255 \text{ OK} \\
 f = 128 &\rightarrow g = 255 - \sqrt{48641} = 255 - 220.5 = 34.5
 \end{aligned}$$

Mid-grey is pushed a long way **down**, to 34.5 — correct, because the input is mostly bright and the target wants mostly dark. The curve is the lower-left quarter of a circle of radius 255.

NEVER INVERT G DIRECTLY

The formal answer is $z = G^{-1}[T(r)]$, and it is tempting to find G^{-1} first. Do not. Setting $G(z) = T(r)$ and solving the resulting algebra for z is the same thing and is almost always easier — a single cube root instead of an inverse function.

9.4 Equalisation against matching

	EQUALISATION	MATCHING (SPECIFICATION)
You supply	Nothing — it is automatic	The target histogram or density
Target shape	Uniform, and you cannot change it	Any shape you like
Transformations built	One — $T(r)$	Two — $T(r)$ and $G(z)$
Final step	Round s_k	Find the z whose $G(z)$ is nearest to s_k ; on a tie take the smallest z
Result in the discrete case	Spread out, but never flat	Close to the target, but never exact
Why not exact	A gray level cannot be split	The same reason — bars move and merge, never divide

EXAM TIP

Do **not** round s_k to a whole number before step 3. The paper says so explicitly, and the reason is that rounding hides the ties — and a tie is exactly where the examiner is watching.

CHAPTER 09 AT A GLANCE

- Three steps every time: equalise the input to get $T(r)$, equalise the target to get $G(z)$, then set $G(z) = T(r)$ and solve for z .
- Discrete: for each s_k pick the z with the nearest $G(z)$; on an exact tie take the **smallest** z .
- Do not round s_k early — rounding hides the ties the examiner is testing.
- Continuous: two integrals and some algebra. Check by confirming the endpoints map to themselves.
- The actual output only **approximates** the target, because a gray level can be moved and merged but never split.

CHAPTER 10

SMOOTHING: MEAN, GAUSSIAN AND MEDIAN

From here on the new value of a pixel depends on a window around it, not just on itself. This chapter settles the three smoothing filters that get examined, the Pascal construction for a Gaussian mask, and the one comparison — mean against median — that has appeared as a standalone question.

10.1 Mask processing

A **mask** (also kernel, template, window) is a small array of coefficients. Lay it over the image, multiply overlapping numbers, add them, divide by the coefficient sum, and write the result at the centre position. Then move on one pixel.

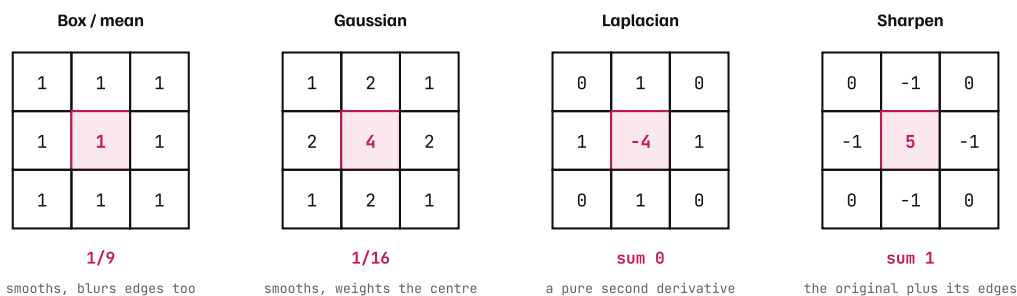
$$g(x, y) = \frac{\sum_s \sum_t w(s, t) f(x + s, y + t)}{\sum_s \sum_t w(s, t)}$$

masks are almost always ODD sized (3x3, 5x5)
so there is a centre

THE ONE-SECOND TEST FOR ANY MASK

Add up its coefficients. Sum **1** (after dividing) means a **smoothing** mask — average brightness is preserved. Sum **0** means a **derivative** mask — no response on a flat region. That one addition tells you which family a mask belongs to and catches most copying errors.

DIAGRAM



the fastest check there is:

coefficients sum to 1 → a smoothing or sharpening mask, brightness is preserved
coefficients sum to 0 → a derivative mask, flat regions give zero response

Fig. 10.1 The three smoothing masks on one 3 × 3 window: the box filter with nine equal weights, the Pascal-Gaussian with weights falling away from a heavy centre, and the median shown as the nine values sorted into a row with the fifth one circled.

10.2 The box filter

All nine coefficients equal, divide by 9. Also called the **arithmetic mean filter**, the **averaging filter** or the **low-pass filter**. Its purpose is blurring and noise reduction.

- Removing small detail before pulling out the large objects.
- Bridging small gaps in lines and curves.
- Reducing noise — the one you will be asked about.

BOTH HALVES, ALWAYS

Averaging smooths out sharp transitions in gray level. Random noise is a sharp transition, so it goes. **But so is an edge**, so edges go soft too. There is no averaging filter that does one without the other. State both halves for the mark.

The **size** of the mask sets the size of the objects that get blended into the background. A large mask is used deliberately to eliminate small objects: anything noticeably smaller than the mask vanishes, anything much bigger survives.

10.3 The weighted average and Pascal's triangle

COMPUTED · TOOLS/MAKE-FIGURES.PY

Mean error against the clean image — mean 15.0, Gaussian 14.3, median 6.2. The median does not average the spike in, it refuses to pick it.

salt and pepper, 8% of pixels



mean 3x3



Gaussian 5x5 (Pascal)



median 3x3



Eight per cent of the pixels set to 0 or 255, then the three smoothers. The mean and the Gaussian spread every spike over their whole window; the median simply never selects one. Mean absolute error against the clean image: mean 15.0, Gaussian 14.3, **median 6.2**.

In a weighted mask the centre is the heaviest coefficient and the others fall off with distance. The reason is stated plainly in the lecture: it is *an attempt to reduce blurring in the smoothing process*. The pixel still gets averaged, but its own value gets the loudest vote, so it is dragged less far.

WORKED EXAMPLE

Generate the 5×5 Gaussian mask from Pascal's triangle.

The binomial coefficients are a discrete approximation to a Gaussian, so the row with five entries is the 1-D kernel:

```

      1
    1 1
  1 2 1
1 3 3 1
1 4 6 4 1    ← five entries, the row we need

```

A Gaussian is **separable**, so the 2-D mask is the outer product of that row with itself:

```

[1]
[4]
[6] x [ 1  4  6  4  1 ]
[4]
[1]

      1   4   6   4   1
      4  16  24  16   4
1/256 x 6  24  36  24   6
      4  16  24  16   4
      1   4   6   4   1

```

The normalising constant. The coefficients must sum to 1 or the image changes brightness:

$$\sum = (1 + 4 + 6 + 4 + 1) \times (1 + 4 + 6 + 4 + 1) = 16 \times 16 = 256$$

The same construction from the row **1 2 1** gives the familiar 3×3 Gaussian, normalised by $4 \times 4 = 16$:

```

      1  2  1
1/16 x 2  4  2
      1  2  1

```

	ARITHMETIC MEAN (BOX)	GAUSSIAN
Weights	All equal — a corner pixel counts as much as the centre	Centre heaviest, falling off smoothly with distance
Blurring	More — distant pixels drag the centre a long way	Less , for the same amount of noise reduction
Artefacts	Sharp mask edges give a sinc frequency response with sidelobes → ringing and a boxy look	The transform of a Gaussian is a Gaussian — no sidelobes, no ringing
Shape	Square — smooths more along the diagonals than along the axes	Isotropic — circularly symmetric, direction-independent

10.4 The median filter

Every filter so far is **linear** — multiply and add. **Order-statistic** filters are **non-linear**: the response comes from **ranking** the pixels in the window, not weighting them.

```
R = median{ z_k | k = 1, 2, ... , n x n }
```

1. sort all $n \times n$ values -- ascending or descending, it makes no difference
2. take the middle one

for 3×3 that is the 5th of the 9 sorted values

THE EASY MARK TO LOSE

The pixel's **own original value is included** in the sort. You are ranking all nine values of a 3×3 window, not the eight neighbours.

10.5 The comparison question

WORKED EXAMPLE

For the window below, compute the box mean, the Gaussian output and the median. Say which you would use on salt-and-pepper noise, and why.

$$\begin{bmatrix} 18 & 22 & 33 \\ 34 & 128 & 24 \\ 22 & 19 & 32 \end{bmatrix}$$

Arithmetic mean — add all nine and divide:

$$\begin{aligned} \sum &= 18 + 22 + 33 + 34 + 128 + 24 + 22 + 19 + 32 = 332 \\ \text{mean} &= \frac{332}{9} = 36.9 \end{aligned}$$

Median — sort all nine, including the centre:

```
sorted : 18, 19, 22, 22, [24], 32, 33, 34, 128
                        ^ the 5th of 9
median = 24
```

mean = 36.9 · median = 24

A cleaner window to see all three at once — a single bright speck on a flat field of 10:

```
10  10  10
10  90  10
10  10  10
```

```
box      : (8 x 10 + 90) / 9 = 170 / 9   = 18.9
gaussian : (10+20+10 +20+360+20 +10+20+10) / 16
          = 480 / 16              = 30.0
median   : sorted 10 10 10 10 [10] 10 10 10 90 = 10
```

Read the three numbers. The box filter smears the speck over the whole window. The Gaussian, because the centre carries weight 4 of 16, actually keeps *more* of the speck — it protects detail, which is exactly what you want for a real feature and exactly what you do not want for noise. The median throws the speck away outright and returns the genuine local value, 10.

The primary difference, in one sentence each.

The mean filter is **linear** — it multiplies and adds, so every input, however wrong, contributes to the output. The median filter is an **order-statistic** filter and therefore **non-linear** — it ranks the values and picks one, so an extreme value is never selected.

For salt-and-pepper noise, use the median, and give both reasons:

1. The 128 (or the 90) is a noise spike. The mean lets it drag the answer up, and worse, that single bad pixel contributes to **all nine** output pixels around it — one speck becomes a grey smudge.
2. The median simply never chooses it. The noise pixel is **removed outright** and replaced by a genuine neighbouring value, so at the same mask size you get far better noise reduction with **considerably less blurring**.

EXAM TIP

Impulse noise, salt-and-pepper noise and shot noise are the same thing: pixels stuck at 0 (pepper) or at $L-1$ (salt). If the question names any of the three, the answer is the median.

CHAPTER 10 AT A GLANCE

- Smoothing mask coefficients sum to **1** after dividing; derivative mask coefficients sum to **0**. One addition tells you which you have.
- The box filter averages equally, reduces noise **and** blurs edges — always both, never one without the other.
- Mask size sets the size of the objects that disappear into the background.
- A Gaussian mask is the outer product of a Pascal row with itself: $1 \ 4 \ 6 \ 4 \ 1$ gives the 5×5 mask, normalised by 256.
- Gaussian versus box: less blurring, no ringing, isotropic — because it is smooth and separable rather than square-edged.
- Median: sort all $n \times n$ values **including the centre** and take the middle. Non-linear, and the right answer for salt-and-pepper noise.

CHAPTER 11

SHARPENING: DERIVATIVES, THE LAPLACIAN AND HIGH BOOST

Smoothing was averaging, which is a kind of integration. Sharpening is the opposite operation — differentiation. This chapter settles what a digital derivative must do, the scan-line calculation that both papers set, the derivation of the Laplacian mask, and the one dial that turns it into high-boost filtering.

11.1 What a digital derivative must do

Derivatives of a digital function are defined in terms of **differences**. Before choosing a definition, the lecture lays down what any sensible one has to satisfy. Learn both columns — it is a favourite short question.

	FIRST DERIVATIVE	SECOND DERIVATIVE
In an area of constant intensity	Must be zero	Must be zero
At the onset of a step or ramp	Must be non-zero	Must be non-zero
At the end of a step or ramp	—	Must be non-zero
Along a ramp of constant slope	Must be non-zero	Must be zero

THE WHOLE DIFFERENCE, IN THAT LAST ROW

A ramp is a steady climb. The first derivative sees a steady non-zero slope all the way up it, so it responds along the whole ramp and gives **thick** edges. The second derivative sees no *change* in the slope, so it reads zero except at the two corners, and gives **thin** edges.

first : $\frac{\partial f}{\partial x} = f(x+1) - f(x)$

second : $\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$

first asks "how fast is the brightness changing here?"

second asks "is the rate of change itself changing?"

11.2 The scan-line question

FROM THE PAPER

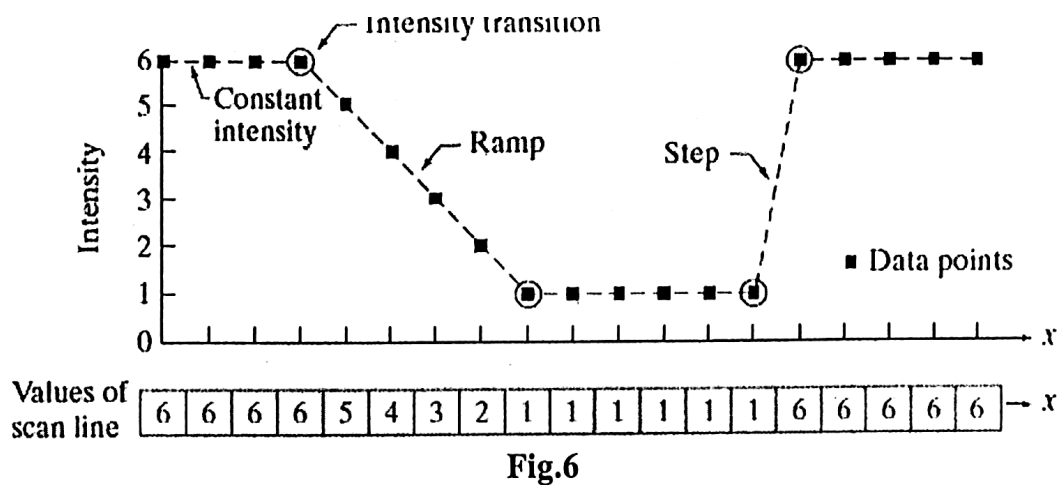
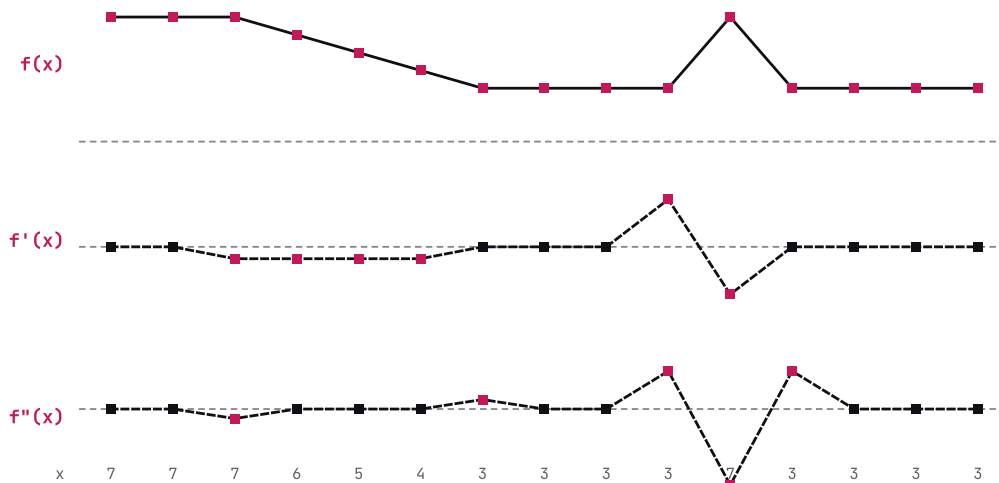


Fig. 11.1 The gray-level profile of a scan line from the September 2025 paper — a constant stretch, a ramp down, another constant stretch and a step back up, with the seven-level values written underneath. Copy the values into a row before you differentiate anything.

DIAGRAM



along the ramp the first derivative is non-zero all the way — a thick edge. The second is zero in the middle and fires only at the two corners — a thin one, and its sign change pins the step to a pixel.

Fig. 11.2 The same profile drawn three times, one above the other on a shared x-axis: the values, then f' showing a wide plateau over the ramp and a single spike at the step, then f'' showing zero through the ramp, a blip at each corner, and the plus-then-minus pair whose zero crossing pins the edge.

WORKED EXAMPLE

Find the first and second derivatives of the scan line, from the second sample through to the penultimate one.

```

index : 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
value : 6  6  6  6  5  4  3  2  1  1  1  1  1  1  6  6  6  6  6
      └-constant-┘ └----ramp----┘ └--constant--┘ └step└const┘

```

First derivative $f'(x) = f(x+1) - f(x)$, over indices 1 to 17:

```

index : 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
f'(x) : 0  0 -1 -1 -1 -1 -1  0  0  0  0  0  5  0  0  0  0

```

Second derivative $f''(x) = f(x+1) + f(x-1) - 2f(x)$, same range:

```

index : 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
f''(x): 0  0 -1  0  0  0  0  1  0  0  0  0  5 -5  0  0  0

```

Show the working for a few — the examiner asks for it:

```

f'(3)   = f(4) - f(3)           = 5 - 6       = -1
f'(13)  = f(14) - f(13)        = 6 - 1        = 5
f''(3)  = f(4) + f(2) - 2f(3)  = 5 + 6 - 12   = -1
f''(8)  = f(9) + f(7) - 2f(8)  = 1 + 2 - 2    = 1
f''(13) = f(14) + f(12) - 2f(13) = 6 + 1 - 2 = 5
f''(14) = f(15) + f(13) - 2f(14) = 6 + 1 - 12 = -5

```

The comments — this is where the marks are.

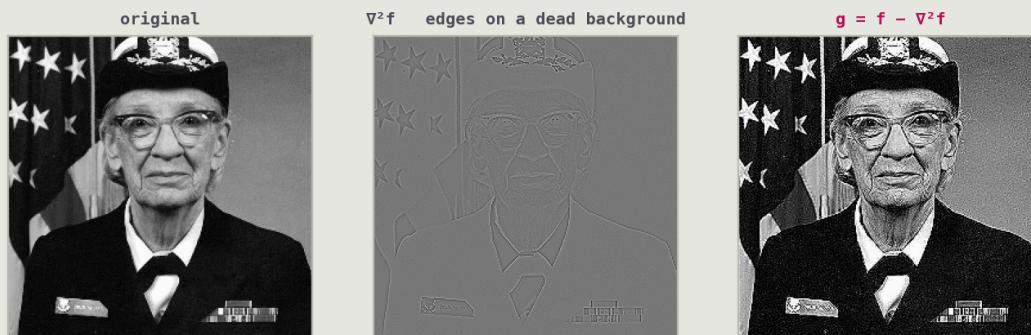
REGION	FIRST DERIVATIVE	SECOND DERIVATIVE
Constant intensity	Zero, as required	Zero, as required
Along the ramp (6 down to 1)	Non-zero throughout — a steady -1 all the way down, so a thick response spanning the entire ramp	Zero in the middle ; non-zero only at the onset (-1) and the end ($+1$), so a thin response marking just the two corners
At the step (1 up to 6)	A single spike of $+5$	A double response, $+5$ then -5 , of opposite sign
Zero crossing	None	The point between the $+5$ and the -5 — it locates the edge to the exact pixel

The conclusion to write out. First derivatives give thicker edges and respond to gradual change. Second derivatives respond far more strongly to fine detail such as isolated points and thin lines, and their zero crossings locate an edge precisely. That is exactly why the **Laplacian** — a second derivative — is the standard sharpening filter, while the **gradient** — a first derivative — is the standard edge detector.

11.3 The Laplacian

COMPUTED · TOOLS/MAKE-FIGURES.PY

The Laplacian on its own is not an image — it is a map of the discontinuities. Add it back and you have the sharpened picture.



The Laplacian on its own is not an image — it is edges on a dead background, because a derivative answers only to change. $g = f - \nabla^2 f$ puts them back onto the original.

We want a filter whose response does not depend on the **direction** of the discontinuity. Such filters are called **isotropic**, meaning **rotation invariant**: rotating then filtering gives the same answer as filtering then rotating. The simplest isotropic derivative operator is the Laplacian.

WORKED EXAMPLE

Derive the 3×3 Laplacian mask from its definition.

$$\text{definition : } \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

each second derivative as a difference :

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

add them :

$$\nabla^2 f = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

Read the coefficients straight off — +1 on each of the four side neighbours, -4 on the centre, 0 on the corners:

$$\begin{array}{ccc} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{array} \quad \text{coefficients sum to 0}$$

with the diagonals included :

$$\begin{array}{ccc} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{array} \quad \text{also sums to 0}$$

Why the Laplacian alone is not an enhanced image. It is a pure derivative operator, so it responds only to **change**. Every region of slowly varying gray gives zero. The result is edges and fine detail sitting on a **dark, featureless background** — a map of the discontinuities, not a picture. The lecture is blunt about it: “but that is not very enhanced!”

The one-pass sharpening mask. Add the edges back onto the original:

$$\begin{aligned} g(x, y) &= f(x, y) - \text{grad}^2 f(x, y) && \text{(centre of the mask is negative)} \\ &= f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)] \\ &= 5 f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)] \end{aligned}$$

$$\begin{array}{ccc} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{array} \quad \text{coefficients sum to 1}$$

The 5 is the 1 from the original plus the 4 from the Laplacian's centre. The coefficients now add to 1, not 0, because this is no longer a pure derivative — it is “the original image, *plus* its edges”, so it must preserve average brightness.

Put a window through it to see the effect:

```

window :  10  10  10
          10  50  10
          10  10  10

grad^2 f = 10 + 10 + 10 + 10 - 4(50) = 40 - 200 = -160
g = f - grad^2 f = 50 - (-160) = 210

or in one pass :  5(50) - (10+10+10+10) = 250 - 40 = 210      same

```

The bright centre has been pushed from 50 to 210. That is sharpening: the difference between a pixel and its surroundings has been exaggerated.

THE SIGN TRAP

Whether the mask centre is -4 or $+4$ decides how you combine the result with the original. **Centre negative** $\rightarrow g = f - \nabla^2 f$. **Centre positive** $\rightarrow g = f + \nabla^2 f$. Both give the same sharpened picture. Say which convention you are using in one line before you start.

EXAM TIP – THE CHECK THAT CATCHES SIGN ERRORS

A **pure derivative** mask has coefficients summing to **0**, so it gives no response on a flat region. A **sharpening** mask has coefficients summing to **1**, because it is the original plus its edges. Add up any mask you write and you know at once which one you have built.

11.4 Unsharp masking and high boost

Unsharp masking comes from the darkroom: subtract a blurred copy of the image from the original, and what is left is the detail. Add that detail back and the picture is sharper. High-boost filtering asks one further question — what if we threw the original in **more than once**?

unsharp mask : $g_mask = f - f_blurred$ (the high pass)
 $g = f + k \cdot g_mask$ $k > 0$

high boost : $g = A \cdot f - \text{low pass}$
 $= (A - 1) f + f - \text{low pass}$
 $= (A - 1) f + \text{high pass}$ $A \geq 1$

masks :

0	-1	0
-1	A+4	-1
0	-1	0

-1	-1	-1
-1	A+8	-1
-1	-1	-1

FOLLOWING THE THREE LINES

Line 1 is the definition: take A copies of the original and subtract the blurred version.

Line 2 splits the A copies into “one copy” plus “ $A-1$ spare copies”. **Line 3** recognises that *original minus low-pass is exactly the high-pass*. So high boost is the sharpened image plus $A-1$ extra helpings of the original. Turning A up fades the picture back in behind the edges.

Its purpose, stated in the lecture, is to **retain some of the low frequency components to aid in the interpretation of the image**. A pure sharpening filter kills the flat regions; high boost keeps enough of them for the picture to still look like a picture. It is also called the **high frequency emphasis filter**.

$A = 1 \rightarrow \text{centre} = 1 + 4 = 5 \rightarrow$ the ordinary Laplacian sharpening mask

$A = 2 \rightarrow \text{centre} = 6$

stronger original, gentler sharpening

$A = 3 \rightarrow \text{centre} = 7$

nearly the original, lightly sharpened

small $A \rightarrow$ strong edges, dark flats

large $A \rightarrow$ nearly the original image

EXAM TIP

Put $A = 1$ into $A + 4$ and you get 5 — the composite mask from the previous section. So high boost is not a new filter, it is the Laplacian sharpener with a dial on it, and $A = 1$ is the setting where the dial does nothing. Saying that is the check that proves you understand it.

CHAPTER 11 AT A GLANCE

- Constant region: both derivatives zero. Along a ramp: first is non-zero (**thick edges**), second is zero (**thin edges** plus a zero crossing).
- $\frac{\partial f}{\partial x} = f(x+1) - f(x)$; $d^2f/dx^2 = f(x+1) + f(x-1) - 2f(x)$.
- At a step the first derivative gives one spike; the second gives + then -, and the **zero crossing** between them locates the edge exactly.
- Laplacian $\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$ — isotropic, mask $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$, sums to 0.
- The Laplacian alone is edges on a dark background. Add it back: $g = f - \nabla^2 f$, composite mask $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$, sums to 1.
- High boost = $A f - \text{low pass} = (A - 1)f + \text{high pass}$, centre $A + 4$ or $A + 8$, $A \geq 1$; $A = 1$ gives back the plain sharpening mask.

CHAPTER 12

THE GRADIENT: ROBERTS, PREWITT AND SOBEL

The first derivative was set aside for sharpening, because the Laplacian does that better. Here it comes back, because for **edge detection** the gradient is what everybody actually uses. This chapter settles the three named masks, the magnitude approximation, and how to read the edge direction off two numbers.

12.1 The gradient vector and its magnitude

$$\nabla f = [G_x G_y]^T = \left[\frac{\partial f}{\partial x} \frac{\partial f}{\partial y} \right]^T$$

$$\text{exact : } M(x, y) = \sqrt{G_x^2 + G_y^2}$$

$$\text{approx : } M(x, y) = |G_x| + |G_y|$$

much cheaper, used everywhere

magnitude → the **STRENGTH** of the edge

orientation → the direction of greatest change, which is **ACROSS** the edge

THE SUBTLETY EXAMINERS LIKE

The **components** G_x and G_y are derivatives, so they are **linear** operators — but the **magnitude is not**, because of the squaring and the square root. The other way round for rotation: the individual partial derivatives are **not** isotropic, but the **magnitude of the gradient vector is**. That is why we always work with the magnitude.

DIAGRAM

the window

10	10	10
10	10	10
50	50	50

the edge runs this way



the gradient points this way

SOBEL

G_x

-1	-2	-1
0	0	0
1	2	1

$$= 160$$

G_y

-1	0	1
-2	0	2
-1	0	1

$$= 0$$

$$M \approx |G_x| + |G_y| = 160 + 0 = 160$$

G_y is zero, so nothing is changing across the rows. All the change is down the columns, so the gradient points down — and since it points across an edge, the edge itself is horizontal.

Fig. 12.1 A dark region meeting a bright region across a horizontal edge, with the gradient vector drawn pointing across it from dark to bright, G_x and G_y shown as its two components, and the Sobel mask pair beside it labelled with which direction each one detects.

12.2 The masks

COMPUTED · TOOLS/MAKE-FIGURES.PY

Gx answers to change down the columns, Gy to change across the rows — and the gradient points across an edge, never along it.



Sobel, applied. $|Gx|$ picks out the horizontal edges because it measures change down the columns, and $|Gy|$ the vertical ones. The sum of the two is the edge map.

label the 3x3 region :

z1	z2	z3	
z4	z5	z6	z5 is the pixel being processed
z7	z8	z9	

Roberts (2x2, works on the DIAGONALS):

$Gx = z9 - z5$	$Gy = z8 - z6$
-1 0	0 -1
0 1	1 0

Prewitt (3x3, all +/-1):

$Gx = (z7+z8+z9) - (z1+z2+z3)$	$Gy = (z3+z6+z9) - (z1+z4+z7)$
-1 -1 -1	-1 0 1
0 0 0	-1 0 1
1 1 1	-1 0 1

Sobel (3x3, a 2 in the middle):

$Gx = (z7+2z8+z9) - (z1+2z2+z3)$	$Gy = (z3+2z6+z9) - (z1+2z4+z7)$
-1 -2 -1	-1 0 1
0 0 0	-2 0 2
1 2 1	-1 0 1

OPERATOR	SIZE	COEFFICIENTS	CHARACTER
Roberts	2×2	± 1 on a diagonal	Cheapest and fastest; very sensitive to noise; no centre pixel, so the edge is offset by half a pixel
Prewitt	3×3	All ± 1	Simple by hand; averages three rows, so steadier than Roberts
Sobel	3×3	± 1 and ± 2	The extra weight on the centre smooths as well as differentiates — the usual choice in practice

THE ONLY DIFFERENCE BETWEEN PREWITT AND SOBEL

The middle coefficient of each row is ± 2 in Sobel, to give more weight to the centre point. That extra weight **smooths as well as differentiates**, which makes Sobel less sensitive to noise. Everything else about the two is identical.

12.3 The worked question

WORKED EXAMPLE

For the window below, compute the gradient magnitude at the centre pixel using the Sobel operator and the $|G_x| + |G_y|$ approximation. State what the two components say about the direction of the edge.

```

window :  z1 z2 z3      10  10  10
           z4 z5 z6    =  10  10  10
           z7 z8 z9      50  50  50
  
```

$$\begin{aligned}
 G_x &= (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3) \\
 &= (50 + 100 + 50) - (10 + 20 + 10) \\
 &= 200 - 40 \\
 &= 160
 \end{aligned}$$

$$\begin{aligned}
 G_y &= (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7) \\
 &= (10 + 20 + 50) - (10 + 20 + 50) \\
 &= 80 - 80 \\
 &= 0
 \end{aligned}$$

$$M = |G_x| + |G_y| = 160 + 0 = 160$$

$$\text{exact : } \sqrt{160^2 + 0} = 160$$

they agree here because $G_y = 0$

Gradient magnitude = 160.

What the components say. G_x is large and G_y is exactly zero, so the brightness is changing rapidly **down the columns** and not at all **across the rows**. The gradient vector therefore points straight down; and since the gradient points *across* an edge, the edge itself runs **horizontally** — which is exactly what the window shows: a flat region of 10s sitting on a flat region of 50s.

The same window with Prewitt, for comparison:

$$\begin{aligned}
 G_x &= (50 + 50 + 50) - (10 + 10 + 10) = 150 - 30 = 120 \\
 G_y &= (10 + 10 + 50) - (10 + 10 + 50) = 70 - 70 = 0 \\
 M &= 120
 \end{aligned}$$

Prewitt gives 120 where Sobel gives 160, on the same edge. The numbers differ because Sobel weights the middle row by 2; the *conclusion* — a horizontal edge, gradient pointing down — is identical.

And Roberts on the same window:

$$Gx = z_9 - z_5 = 50 - 10 = 40$$

$$Gy = z_8 - z_6 = 50 - 10 = 40$$

$$M = |40| + |40| = 80$$

Rotate the window so the edge runs the other way and the two components swap:

$$\text{window} = \begin{bmatrix} 10 & 10 & 50 \\ 10 & 10 & 50 \\ 10 & 10 & 50 \end{bmatrix}$$

$$\text{Sobel : } Gx = (10 + 20 + 50) - (10 + 20 + 50) = 0$$

$$Gy = (50 + 100 + 50) - (10 + 20 + 10) = 160$$

$$M = 160$$

a VERTICAL edge

EXAM TIP – THE FREE CHECK

The coefficients of **every** gradient mask add up to **0**, so each gives a response of 0 in an area of constant gray level. Add up any mask you write down. If it does not come to zero, you have copied it wrong, and that takes five seconds to find out.

DO NOT MIX UP THE TWO DIRECTIONS

Gx is the derivative **in x**, and for the usual convention x runs down the rows. So Gx is computed from the **top and bottom rows** and it detects **horizontal** edges. A large Gx means the edge runs horizontally, not vertically. Write that sentence out before you draw any conclusion about direction.

Finally, the method for a full question that asks you to sharpen an image with all three operators: multiply the 3×3 window by the Gx mask and add, do the same with the Gy mask, then take $|Gx| + |Gy|$. That is the output pixel. Repeat for every position, remembering that the border needs padding or must be left out.

CHAPTER 12 AT A GLANCE

- $\nabla f = [G_x, G_y]$, magnitude $\sqrt{G_x^2 + G_y^2} \approx |G_x| + |G_y|$ — the approximation is what you use by hand.
- The components are linear but the magnitude is not; the components are not isotropic but the magnitude is.
- Roberts 2×2 on the diagonals · Prewitt 3×3 all ± 1 · Sobel 3×3 with a 2 in the middle for extra smoothing.
- All gradient mask coefficients sum to **zero** — add them up as a free check.
- A large G_x with $G_y = 0$ means a **horizontal** edge, because the gradient points across the edge, not along it.

THE WHOLE SUBJECT IN ONE SCREEN

EVERYTHING, CONDENSED

01 THE DIGITAL IMAGE: SAMPLING, QUANTISATION, RESOLUTION

- Sampling makes the **coordinates** discrete; quantisation makes the **amplitude** discrete. Sampling fixes pixel count, quantisation fixes shade count.
- $L = 2^k$ gray levels, values in $[0, L-1]$; storage is $b = M \times N \times k$ bits, and 8 bits = 1 byte.
- Too few samples → blur, then the **checkerboard effect**. Too few levels → **false contouring**, seen first in smooth areas.
- Halving each side quarters the pixel count — 1024 to 256 loses 93.75 % of the pixels.
- Detailed pictures survive on fewer gray levels; smooth pictures need more.

02 BIT PLANES

- Bit-plane slicing splits an 8-bit image into **eight binary images**; plane 8 is the MSB with weight 128.
 - Higher planes hold the visually significant shape; lower planes hold fine detail and look like speckle.
 - To reconstruct: write the binary, keep the wanted bits, zero the rest, add the surviving weights.
 - Keeping planes 8 and 7 gives only **four** possible outputs — 0, 64, 128, 192; keeping n planes gives 2^n values.
 - Say which numbering convention you use — the MSB is the “8th” plane here, the “7th” in books that count from 0.
-

03 NEIGHBOURS, ADJACENCY, PATHS AND DISTANCE

- N_4 = four sides, N_D = four corners, N_8 = all eight; a border pixel has fewer than eight.
- Adjacency needs **both** values in V and the right position; m-adjacency allows a diagonal only if $N_4(p) \cap N_4(q)$ holds nothing from V .
- m-adjacency exists to make the path **unique**; it never makes a path shorter than the 8-path.
- $D_4 = |x - s| + |y - t|$, $D_8 = \max(|x - s|, |y - t|)$, $D_e = \sqrt{(x-s)^2 + (y-t)^2}$, and always $D_8 \leq D_e \leq D_4$.
- D_4 and D_8 use coordinates only; D_m is a real path length, so it changes when the pixels in between change.
- A distance formula can return a number even when no path of that type exists — check the neighbours of q before you start walking.

04 2-D SIGNALS AND CONVOLUTION

- Size first: $M_1 \times M_2$ convolved with $N_1 \times N_2$ gives $(M_1+N_1-1) \times (M_2+N_2-1)$. Add and subtract one; never multiply.
- The whole method: $x(a, b) \cdot h(c, d)$ lands at $y(a+c, b+d)$. Place every product, then add up each box.
- Two free checks — $y(0, 0) = x(0, 0)h(0, 0)$, and $\text{sum}(y) = \text{sum}(x) \times \text{sum}(h)$.
- Mask filtering keeps the image size, so the border needs **zero padding** — write the padded array out before calculating.
- Zero padding always darkens the edges; that is the cost of the simplest border rule.

05 THE 2-D FOURIER TRANSFORM

- An impulse **pair** transforms to a **cosine**; a cosine transforms to an impulse pair. That one fact answers the standard question in either direction.
- $\delta(x, y) \rightarrow F(u, v) = 1$ — a point in space is flat in frequency.
- $0.5[\delta(x-a, y) + \delta(x+a, y)] \rightarrow \cos(2\pi ua)$ — bands of period $\frac{1}{2a}$; bigger a means tighter bands.
- A sinusoid varying along x gives vertical stripes and impulses on the **u-axis** at $\pm$ its frequency, plus DC.
- Convolution theorem: convolving two sinusoids at different frequencies leaves only the DC term — a uniform grey field.

06 INTENSITY TRANSFORMATIONS

- $s = T(r)$ — point processing, neighbourhood 1×1 . Above the diagonal brightens, below darkens, steeper means more contrast.
- Negative $s = L - 1 - r$; it suits **bright detail inside large dark regions**, such as an X-ray.
- Log $s = c \log(1 + r)$ expands dark, compresses bright; its real job is **dynamic-range compression**, as for a Fourier spectrum.
- Power law $s = c r^\gamma$: $\gamma < 1$ brightens a dark image, $\gamma > 1$ darkens a washed-out one. Correct a monitor of gamma 2.5 with $1/2.5 = 0.4$.
- Piecewise linear covers contrast stretching, thresholding and level slicing — any shape, but you must supply the corner points.

07 THE HISTOGRAM, ITS STATISTICS AND LOCAL ENHANCEMENT

- Histogram $h(r_k) = n_k$, normalised $p(r_k) = \frac{n_k}{MN}$ summing to 1; it keeps no position information.
- Mean two ways — $(1/MN) \sum \sum f(x, y)$ or $\sum r_i p(r_i)$ — and they must agree, because grouping cannot change a sum.
- Mean measures **brightness**, variance $\sum (r_i - m)^2 p(r_i)$ measures **contrast**.
- Global equalisation misses small regions; local equalisation builds a fresh transformation inside a window and maps **only the centre pixel**.
- Sliding window: one transformation per pixel, smooth, cheap because the histogram updates incrementally. Block method: fast but leaves **blocking artefacts**.
- Adaptive rule: multiply by C only where the patch is dark ($m_S \leq k \leq m_G$) and faint ($k_1 \leq s \leq k_2$).

08 HISTOGRAM EQUALISATION

- $s_k = (L-1) \times (\text{cumulative count up to } k) / MN$, then round to the nearest gray level.
- The CDF is chosen because it is single-valued, monotonically increasing, and maps $[0, L-1]$ onto $[0, L-1]$.
- Draw the table: r_k, n_k , cumulative, s_k , rounded. Then the mapping, then the output image, then the output histogram.
- The result is **never flat** — bars can be moved and merged but never split, so only the continuous case gives a uniform histogram.
- Checks: counts sum to MN both times, and the last s_k must equal exactly $L-1$.

09 HISTOGRAM MATCHING

- Three steps every time: equalise the input to get $T(r)$, equalise the target to get $G(z)$, then set $G(z) = T(r)$ and solve for z .
- Discrete: for each s_k pick the z with the nearest $G(z)$; on an exact tie take the **smallest** z .
- Do not round s_k early — rounding hides the ties the examiner is testing.
- Continuous: two integrals and some algebra. Check by confirming the endpoints map to themselves.
- The actual output only **approximates** the target, because a gray level can be moved and merged but never split.

10 SMOOTHING: MEAN, GAUSSIAN AND MEDIAN

- Smoothing mask coefficients sum to **1** after dividing; derivative mask coefficients sum to **0**. One addition tells you which you have.
- The box filter averages equally, reduces noise **and** blurs edges — always both, never one without the other.
- Mask size sets the size of the objects that disappear into the background.
- A Gaussian mask is the outer product of a Pascal row with itself: $1 \ 4 \ 6 \ 4 \ 1$ gives the 5×5 mask, normalised by 256.
- Gaussian versus box: less blurring, no ringing, isotropic — because it is smooth and separable rather than square-edged.
- Median: sort all $n \times n$ values **including the centre** and take the middle. Non-linear, and the right answer for salt-and-pepper noise.

11 SHARPENING: DERIVATIVES, THE LAPLACIAN AND HIGH BOOST

- Constant region: both derivatives zero. Along a ramp: first is non-zero (**thick** edges), second is zero (**thin** edges plus a zero crossing).
- $\frac{\partial f}{\partial x} = f(x+1) - f(x)$; $d^2f/dx^2 = f(x+1) + f(x-1) - 2f(x)$.
- At a step the first derivative gives one spike; the second gives + then -, and the **zero crossing** between them locates the edge exactly.
- Laplacian $\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$ — isotropic, mask $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$, sums to 0.
- The Laplacian alone is edges on a dark background. Add it back: $g = f - \nabla^2 f$, composite mask $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$, sums to 1.
- High boost = $A \cdot f - \text{low pass} = (A - 1)f + \text{high pass}$, centre $A + 4$ or $A + 8$, $A \geq 1$; $A = 1$ gives back the plain sharpening mask.

12 THE GRADIENT: ROBERTS, PREWITT AND SOBEL

- $\nabla f = [G_x, G_y]$, magnitude $\sqrt{G_x^2 + G_y^2} \approx |G_x| + |G_y|$ — the approximation is what you use by hand.
 - The components are linear but the magnitude is not; the components are not isotropic but the magnitude is.
 - Roberts 2×2 on the diagonals · Prewitt 3×3 all ± 1 · Sobel 3×3 with a 2 in the middle for extra smoothing.
 - All gradient mask coefficients sum to **zero** — add them up as a free check.
 - A large G_x with $G_y = 0$ means a **horizontal** edge, because the gradient points across the edge, not along it.
-

That is the entire revision set — 64 lines. If a line does not make sense, the chapter it came from is one tap away, and the **full course** is behind that.